

Predictive Memory Paging System

Implementation Guide: Stages 1-5

Complete walkthrough of the data pipeline, prediction model, memory engine, and decision engine, with final code for every cell and all results.

Contents

- **Stage 1 — Data acquisition** (Parts 0-3): Colab setup, synthetic trace generators, real benchmark traces, page-granularity conversion
 - **Stage 2 — Normalisation** (Part 4): the `Trace` class, time-ordered splitting, streaming interface
 - **Stage 3 — Prediction model** (Parts 7-13): Markov / stride / hybrid predictors, evaluation metrics, hyperparameter freezing, memory footprint, capacity sweep, phase adaptation
 - **Stage 4 — Memory engine** (Parts 14-20): page/frame tables, FIFO / LRU / Clock / OPT, working-set-scaled frame sweep, correctness verification, headroom analysis
 - **Stage 5 — Decision engine** (Parts 21-29): prediction/decision separation, six deciders, shadow-engine fault-delta feedback, ablation results, findings, limitations
 - **Appendix A** — common errors and fixes
 - **Appendix B** — key numbers at a glance
-

STAGES 1 & 2: The Data Pipeline

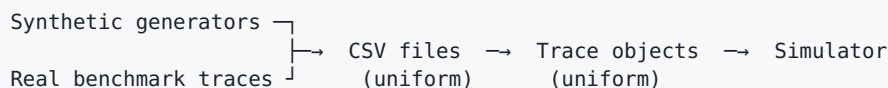
Covers getting the data and preparing it for the simulator.

Part 0: Why this pipeline exists

Before any code, the shape of what you're building:

Your simulator needs **a sequence of page numbers**. That's it. Nothing more complicated. Every access in the sequence is one "the program wants this page" event, and your simulator's job is to decide whether that page was already in RAM.

Everything in Stages 1 and 2 exists to produce that sequence, from two different sources, in one uniform format.



The key principle: **by the time data reaches your simulator, it must be impossible to tell where it came from**. A trace from a SPEC benchmark and a trace you generated in ten lines of Python should look identical to the engine. This is what lets you swap datasets freely and compare results fairly.

Part 1: Colab setup

The problem you will hit constantly

Google Colab wipes everything in `/content` whenever the runtime restarts or disconnects. Variables, functions, cloned repos, downloaded files — all gone. Only `/content/drive/MyDrive/...` survives.

This causes the `NameError: name 'X' is not defined` errors that will otherwise waste your time.

The solution: one idempotent setup cell

Put this at the very top of your notebook. Run it after every reconnect. It is safe to run repeatedly — it skips work already done.

```
# ===== CELL 1: SETUP — run first every session =====
from google.colab import drive
drive.mount('/content/drive')

import os, csv, gzip, lzma, json
```

```
# --- paths (Drive = permanent, /content = temporary) ---
BASE = '/content/drive/MyDrive/paging_sim'
RAW = f'{BASE}/raw'          # compressed originals
OUT = f'{BASE}/traces'       # converted CSVs — what everything else reads
os.makedirs(RAW, exist_ok=True)
os.makedirs(OUT, exist_ok=True)

# --- simulation constants ---
PAGE_SHIFT = 12               # 4KB pages (2^12 = 4096)
MAX_ACCESSES = 500_000        # cap per trace

print('setup done')
```

What each path is for

Path	Survives restart?	Holds
/content/...	No	Downloads in progress, cloned repos
{RAW}	Yes	Original .xz compressed traces
{OUT}	Yes	Converted CSVs — the important output

Download to `/content` first because writing through the Drive mount is slow, then copy the final small files to Drive.

Part 2 (Stage 1a): Synthetic traces

Why generate fake data at all

Because you know the answer in advance. If your classifier can't correctly identify a pure sequential trace that you generated yourself, the classifier is broken — no ambiguity, no debugging guesswork. Synthetic traces are your **controlled experiments**.

Real traces tell you whether your system is useful. Synthetic traces tell you whether it's *correct*.

The generators

```
# ===== CELL 2: SYNTHETIC GENERATORS =====
import numpy as np

def sequential(n=50000, start=0, stride=1, n_pages=10000):
    """1, 2, 3, 4... — the easiest possible pattern."""
    return [(start + i*stride) % n_pages for i in range(n)]

def strided(n=50000, stride=8, n_pages=10000):
    """0, 8, 16, 24... — regular but with gaps."""
    return sequential(n, 0, stride, n_pages)

def cyclic(n=50000, loop_len=20, start=100):
    """A fixed loop repeated forever: 100..119, 100..119, ..."""
    loop = [start + i for i in range(loop_len)]
    return [loop[i % loop_len] for i in range(n)]

def clustered(n=50000, cluster_size=10, dwell=500, n_pages=10000, seed=0):
    """Bounces randomly inside a small neighbourhood, then jumps elsewhere."""
    rng = np.random.default_rng(seed)
    trace, i = [], 0
```

```

while i < n:
    base = rng.integers(0, n_pages - cluster_size)
    for _ in range(min(dwell, n - i)):
        trace.append(int(base + rng.integers(0, cluster_size)))
        i += 1
return trace

def uniform_random(n=50000, n_pages=10000, seed=0):
    """No structure at all – the worst case for any predictor."""
    rng = np.random.default_rng(seed)
    return rng.integers(0, n_pages, size=n).tolist()

def phase_changing(n=60000, seed=0):
    """Sequential -> cyclic -> random. Tests the ADAPTIVE layer."""
    third = n // 3
    return (sequential(third, start=0, stride=1)
            + cyclic(third, loop_len=25, start=5000)
            + uniform_random(n - 2*third, seed=seed))

```

What each one is testing

Generator	Pattern	What it proves
sequential	1,2,3,4	Stride detector works at all
strided	0,8,16,24	Stride detector handles gaps $\neq 1$
cyclic	loop repeats	Markov chain learns recurring transitions
clustered	local bouncing, then jumps	Handles locality without strict order
uniform_random	noise	System correctly gives up and stops prefetching
phase_changing	all three in sequence	Adaptation actually happens

`phase_changing` is the most important one. It is the only trace that can demonstrate the "adaptive" claim in your project title. A system with a fixed strategy will do well on one third of it and badly on the rest; an adaptive system should recover after each transition.

Note on `seed`

Every random generator takes a `seed`. Same seed = same output, every time. This makes your experiments reproducible, which reviewers check. Always pass seeds explicitly and record them.

Saving them

```

# ===== CELL 3: GENERATE AND SAVE =====
SYNTH = {
    'sequential': sequential(),
    'strided': strided(),
    'cyclic': cyclic(),
    'clustered': clustered(),
    'random': uniform_random(),
    'phase_changing': phase_changing(),
}

for name, trace in SYNTH.items():
    path = f'{OUT}/{name}.csv'
    with open(path, 'w', newline='') as f:
        w = csv.writer(f)
        w.writerow(['access_id', 'page_id'])
        w.writerows(enumerate(trace))
    print(f'{name:16s} {len(trace):7d} accesses -> {path}')

```

`enumerate(trace)` produces `(0, page0), (1, page1), ...` — which is exactly the two-column format you want, with no extra work.

Verifying them

```
# ===== CELL 4: VISUAL CHECK =====
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 3, figsize=(15, 6))
for ax, (name, trace) in zip(axes.flat, SYNTH.items()):
    ax.plot(trace[:600], lw=0.6)
    ax.set_title(name); ax.set_xlabel('access #'); ax.set_ylabel('page')
plt.tight_layout(); plt.show()

for name, trace in SYNTH.items():
    deltas = np.diff(trace[:5000])
    print(f'{name:16s} unique pages={len(set(trace)):6d} '
          f'unique deltas={len(set(deltas.tolist())):5d}')
```

What correct output looks like:

- `sequential` — a straight diagonal line; **exactly 1 unique delta**
- `strided` — also diagonal, steeper; 1 unique delta
- `cyclic` — tight sawtooth; very few unique deltas
- `clustered` — flat plateaus that jump; moderate delta count
- `random` — noise filling the plot; thousands of unique deltas
- `phase_changing` — visibly changes character twice

If `sequential` reports more than 1 unique delta, your generator has a bug. Fix it before going further — a broken generator silently poisons every experiment downstream.

Part 3 (Stage 1b): Real traces

What a trace actually is

A recording of what a real program did to memory, in order. Someone ran a real benchmark on a real machine with a tool watching every memory access and wrote each one down.

Think of it as a security camera log for memory: "at this moment, the program touched this location." Millions of entries, chronological.

Why this is useful: you can replay it through *your* simulator, as many times as you want, with different settings, and get comparable results — without needing the original program, its input data, or its machine.

Getting the download links

```
# ===== CELL 5: GET TRACE URLS =====
if not os.path.exists('ChampSim'):
    !git clone -q https://github.com/Quangmire/ChampSim.git

URLS = {}
```

```

with open('ChampSim/download_links') as f:
    for line in f:
        p = line.split()
        if len(p) == 2 and 'LoadTraces' in p[0]:
            name = p[0].split('/')[1].replace('.txt.xz', '')
            URLS[name] = p[1]

with open(f'{BASE}/trace_urls.json', 'w') as f:
    json.dump(URLS, f, indent=2)

print(f'{len(URLS)} load traces available')

```

Critical: LoadTraces vs ChampSimTraces

The repo lists two kinds of file. You want **only** LoadTraces :

Folder	Format	Use
LoadTraces	Plain text CSV, .txt.xz	What you want — readable, parseable in Python
ChampSimTraces	Binary, .gz	For the C++ ChampSim simulator only. Ignore.

The `if 'LoadTraces' in p[0]` filter is what enforces this.

The trace file format

A LoadTrace line looks like:

```
3, 13, fd fd3a8c1e00, 401a26, 0
```

Column	Index	Example	Meaning
Unique Instr Id	0	3	Which instruction number in execution
Cycle Count	1	13	When it happened (CPU cycles)
Load Address	2	fd fd3a8c1e00	The memory location — the only column you need. Hex.
Instruction Pointer	3	401a26	Where in the <i>code</i> the instruction lives
LLC hit/miss	4	0	Found in last-level cache (1) or not (0)

Important caveat: these are LLC accesses

A CPU has layers: L1 cache → L2 → L3/LLC → RAM → disk. Most memory accesses hit L1 and never travel further.

This trace only records accesses that reached the LLC — they already missed L1 and L2. The easy, repetitive accesses were absorbed upstream. What's left is the harder, less predictable residue.

Two consequences: - Your reuse numbers will look lower than a full memory trace. **Expected, not a bug.** - The remaining pattern is genuinely the difficult part — which is why prefetching research uses these traces.

State this explicitly in your methodology section. You're applying an OS-page-granularity study to cache-level traces. It's a defensible approximation, but a reviewer will notice if you don't mention it.

Choosing which benchmarks

```
# ===== CELL 6: PICK YOUR SUITE =====
PICK = [
    '410.bwaves-s0',    # numerical/CFD — regular, strided
    '437.leslie3d-s0',  # CFD — regular but more complex
    'bfs-3',            # graph breadth-first search — irregular
    'pr-3',             # PageRank — irregular
]
```

The benchmarks available

Regular / numerical — array-heavy, marches through memory in order: - 410.bwaves — computational fluid dynamics - 437.leslie3d — CFD, more complex access structure - 433.milc — quantum chromodynamics - 462.libquantum — very regular, near-pure streaming

Irregular / graph — follow pointers between nodes; next address depends on *data*, not arithmetic: - bfs — breadth-first search - pr — PageRank - cc — connected components - sssp — single-source shortest path - bc — betweenness centrality

Stress case: - 429.mcf — combinatorial optimization, notoriously memory-hostile, large files

Why contrast matters more than quantity

Four benchmarks that all behave identically tell you nothing. One regular + one irregular tells a real story: your stride predictor should excel on bwaves and fail on bfs, while the Markov chain may find structure where stride detection can't. **Explaining why is a genuine finding.**

The -s0/-s1/-3/-14 suffixes are different sampled segments of the *same* program. Good for consistency checks, weak for diversity. Pick different benchmarks, not different samples.

The converter

```
# ===== CELL 7: CONVERTER =====
def _open(path):
    """Transparently handle .xz, .gz, or plain text."""
    if path.endswith('.xz'): return lzma.open(path, 'rt')
    if path.endswith('.gz'): return gzip.open(path, 'rt')
    return open(path, 'rt')

def convert(in_path, out_path, addr_col=2, is_hex=True,
            delimiter=',', limit=MAX_ACCESSES):
    n, skipped = 0, 0
    with _open(in_path) as fin, open(out_path, 'w', newline='') as fout:
        w = csv.writer(fout)
        w.writerow(['access_id', 'page_id'])
        for line in fin:
            parts = line.strip().split(delimiter)
            if len(parts) <= addr_col:
                skipped += 1
                continue
            try:
                addr = int(parts[addr_col].strip(), 16 if is_hex else 10)
            except ValueError:
                skipped += 1
                continue
            w.writerow([n, addr >> PAGE_SHIFT])
            n += 1
            if n >= limit:
```

```

        break
    return n, skipped

```

Line by line

`_open()` — **streaming decompression**. The `.xz` file may be hundreds of MB compressed and gigabytes raw. `lzma.open(path, 'rt')` reads it as text *on the fly* without ever writing the decompressed version to disk. Essential on Colab's limited storage.

`line.strip().split(',')` — turns `"3, 13, fd3a8c1e00, 401a26, 0"` into `['3', ' 13', ' fd3a8c1e00', ' 401a26', ' 0']`. Note the leading spaces; `.strip()` on the field handles them.

`parts[addr_col]` with `addr_col=2` grabs the Load Address. The other four columns are discarded — your simulator doesn't need them.

`int(raw, 16)` — the `16` says "this is base-16 (hex)". Converts `'fd3a8c1e00'` to `279441668497408`.

`addr >> PAGE_SHIFT` — **this is the conceptual heart of the whole conversion**.

The page shift explained

`>> 12` discards the bottom 12 bits, which is dividing by 4096 ($2^{12} = 4096$). 4096 bytes = 4KB = one memory page.

Your project is about *pages*, not bytes. Two accesses to byte 1000 and byte 2000 are different addresses but live in the **same page** — so to your simulator they're the same thing, and the second is a **hit**.

Concretely, from a real trace:

```

fd3a8c1e00 >> 12 -> page 68222672
fd3a8c2800 >> 12 -> page 68222672  <- same page!

```

Those two accesses are 2560 bytes apart — different addresses, identical page.

Without the shift, your simulator would treat them as unrelated and record two faults instead of one hit. The shift is what turns a raw address log into a *paging* workload. Get this wrong and every number you produce is meaningless.

Downloading and converting

```

# ===== CELL 8: DOWNLOAD + CONVERT =====
for name in PICK:
    out_csv = f'{OUT}/{name}.csv'
    if os.path.exists(out_csv):
        print(f'{name:18s} already converted, skipping')
        continue

    local = f'/content/{name}.txt.xz'
    if not os.path.exists(local):
        print(f'downloading {name} ...')
        !wget -q -O "{local}" "{URLS[name]}"

    # verify it's real xz, not an HTML error page
    try:
        with lzma.open(local, 'rt') as f:

```

```

        first = f.readline().strip()
    except Exception as e:
        print(f'{name}: BAD DOWNLOAD ({e})')
        continue

    mb = os.path.getsize(local) / 1e6
    n, skipped = convert(local, out_csv)
    print(f'{name:18s} {mb:6.0f} MB -> {n:7d} accesses, {skipped} skipped')

!ls -lh {OUT}/

```

Why the verification step matters

Box sometimes returns an **HTML error page** instead of the file. It downloads "successfully" — right filename, non-zero size — but it's garbage. The `lzma.open()` test catches this immediately instead of letting you build on corrupt data.

Watch `skipped`

- 0 or a handful → fine (header/blank lines)
- Thousands → your `delimiter` or `addr_col` is wrong. **Stop and fix before continuing.**

Free the disk

```

# ===== CELL 9: CLEANUP =====
!rm -f /content/*.txt.xz
!df -h /content | tail -1

```

The CSVs are on Drive; the compressed originals aren't needed again.

Characterizing the real traces

```

# ===== CELL 10: CHARACTERIZE =====
import pandas as pd

stats = []
for name in PICK:
    p = pd.read_csv(f'{OUT}/{name}.csv')['page_id'].values
    d = np.diff(p)
    vc = pd.Series(d).value_counts()
    stats.append({
        'trace': name,
        'accesses': len(p),
        'unique_pages': len(np.unique(p)),
        'reuse': round(len(p)/len(np.unique(p)), 2),
        'unique_deltas': len(vc),
        'top_delta': int(vc.index[0]),
        'top_delta_share': round(vc.iloc[0]/len(d), 3),
        'top5_share': round(vc.iloc[:5].sum()/len(d), 3),
    })

df = pd.DataFrame(stats)
display(df)
df.to_csv(f'{BASE}/trace_characteristics.csv', index=False)

```

Reading the metrics

Metric	Meaning	What it tells you
reuse	accesses ÷ unique pages	How often pages get revisited. >1 means locality exists.
unique_deltas	distinct gaps between consecutive accesses	Low = regular. High = irregular.
top_delta_share	fraction of transitions using the single most common gap	High = one dominant stride
top5_share	fraction covered by the top 5 gaps	The headline regularity number

What you want to see:

- `bwaves`, `leslie3d` → `top5_share` high (>0.5) — regular, strided
- `bfs`, `pr` → `top5_share` low (<0.2) — irregular, pointer-chasing

That gap is the entire justification for your adaptive design. If all four traces look identical, swap one out for a more different benchmark.

This table goes directly into your paper's workload description section — reviewers expect exactly this kind of quantitative benchmark characterization.

Part 4 (Stage 2): Normalizing and feeding

Stage 1 produced CSV files. Stage 2 turns them into a **uniform object** that the simulator consumes, and enforces the rules that keep your experiments honest.

The four principles

1. **One format.** Synthetic and real become the same thing: an ordered sequence of page IDs. The engine never knows which is which.
2. **Deltas precomputed.** The gap between consecutive accesses, calculated once here rather than millions of times during simulation.
3. **Split in time order — never shuffle.** Memory traces are sequences; shuffling destroys the structure you're modeling.
4. **Stream, don't batch.** The engine pulls one access at a time, as a real system would. No peeking ahead.

The Trace class

```
# ===== CELL 11: TRACE LOADER =====
import pandas as pd, numpy as np

class Trace:
    """One workload, normalized. Knows nothing about where it came from."""

    def __init__(self, name, pages, train_frac=0.25):
        self.name = name
        self.pages = np.asarray(pages, dtype=np.int64)
        self.deltas = np.diff(self.pages, prepend=self.pages[0])
        self.split = int(len(self.pages) * train_frac)
```

```

# --- the two views ---
@property
def train(self):
    """Warm-up portion. Model MAY learn from this."""
    return self.pages[:self.split]

@property
def test(self):
    """Evaluation portion. Metrics measured ONLY here."""
    return self.pages[self.split:]

# --- streaming interface ---
def stream(self, portion='all'):
    """Yield (index, page, delta) one at a time. No lookahead."""
    src = {'all': (0, len(self.pages)),
           'train': (0, self.split),
           'test': (self.split, len(self.pages))}[portion]
    for i in range(*src):
        yield i, int(self.pages[i]), int(self.deltas[i])

def __len__(self):
    return len(self.pages)

def __repr__(self):
    return (f'<Trace {self.name}: {len(self)} accesses, '
            f'{len(np.unique(self.pages))} unique pages, '
            f'split@{self.split}>')

def load_trace(name, trace_dir=None, limit=None, train_frac=0.25):
    d = trace_dir or OUT # resolved when CALLED, not when defined
    df = pd.read_csv(f'{d}/{name}.csv')
    pages = df['page_id'].values
    if limit:
        pages = pages[:limit]
    return Trace(name, pages, train_frac)

print('Trace loader ready')

```

Design decisions explained

`np.diff(pages, prepend=pages[0])` — computes gaps between consecutive accesses. The `prepend` makes `deltas[0] == 0` so the `deltas` array has the **same length** as the `pages` array. Without it you'd be off by one, and off-by-one errors in array indexing are painful to debug later.

train / test as properties — they're just slices, computed on access. No data is copied. The names make the contract explicit: the model may learn from `train`, but metrics are only reported on `test`.

stream() is a generator — it `yields` one access at a time. This isn't just tidiness. The engine physically cannot look ahead because it never holds the full array. When you later write `OPT` (which *is* allowed to see the future), the asymmetry becomes visible in the code: `OPT` gets the full array, everything else gets `stream()`. A reviewer reading your code sees immediately that only `OPT` cheats.

trace_dir=None then d = trace_dir or OUT — if you wrote `trace_dir=OUT` in the signature, Python evaluates `OUT` at *definition* time and the cell fails with `NameError` if setup hasn't run. Resolving inside the function means it's only needed when you actually load something.

train_frac is a parameter, not a constant — different traces need different warm-up lengths. A Markov table on an irregular graph benchmark needs more observations before it's useful than one on bwaves. Making it configurable lets you test sensitivity to that choice, which is a legitimate experiment to report.

Registering all traces

```
# ===== CELL 12: REGISTER =====
SYNTHETIC = ['sequential', 'strided', 'cyclic', 'clustered', 'random', 'phase_changing']
REAL      = PICK

ALL_TRACES = [n for n in SYNTHETIC + REAL
               if os.path.exists(f'{OUT}/{n}.csv')]

print(f'{len(ALL_TRACES)} traces available:')
for n in ALL_TRACES:
    print(' ', n)
```

The `os.path.exists` filter means missing traces are silently skipped rather than crashing later — useful while you're still assembling the suite.

Verification — the cell that catches real bugs

```
# ===== CELL 13: VERIFY =====
t = load_trace(ALL_TRACES[0], limit=50_000)
print(t)

# 1. lengths line up
assert len(t.pages) == len(t.deltas), 'delta/page length mismatch'
assert len(t.train) + len(t.test) == len(t), 'split loses accesses'

# 2. split is time-ordered, NOT shuffled
assert np.array_equal(np.concatenate([t.train, t.test]), t.pages), \
    'split reordered the sequence!'

# 3. deltas reconstruct the sequence
recon = np.cumsum(t.deltas) + t.pages[0] - t.deltas[0]
assert np.array_equal(recon, t.pages), 'deltas inconsistent'

# 4. streaming matches the array
streamed = [p for _, p, _ in t.stream()]
assert streamed == t.pages.tolist(), 'stream != pages'

print(f'train: {len(t.train)} test: {len(t.test)}')
print(f'first 10 pages : {t.pages[:10].tolist()}')
print(f'first 10 deltas: {t.deltas[:10].tolist()}')
print('all checks passed')
```

Why check #2 is the most important

Shuffling a memory trace destroys the temporal structure you're trying to model. It's an easy mistake if you ever reach for a generic train/test split helper — most ML libraries shuffle **by default**.

This assertion makes it impossible to do silently. If someone (including future you) swaps in `sklearn.model_selection.train_test_split`, this line fails immediately with a clear message.

Why check #3 matters

`np.cumsum(deltas)` rebuilds the original sequence from the gaps. If it doesn't match, your delta computation is wrong — and since the classifier and stride predictor both work on deltas, that would corrupt everything downstream while still *looking* plausible.

Loading everything

```
# ===== CELL 14: LOAD ALL =====
TRACES = {n: load_trace(n, limit=200_000) for n in ALL_TRACES}

rows = []
for n, t in TRACES.items():
    rows.append({
        'trace': n,
        'accesses': len(t),
        'unique': len(np.unique(t.pages)),
        'train': len(t.train),
        'test': len(t.test),
        'kind': 'real' if n in REAL else 'synthetic',
    })
display(pd.DataFrame(rows))
```

Part 5: Notebook structure

Organize your notebook in this order so `Runtime -> Run all` always works after a reconnect:

Cell	Contents	Idempotent?
1	SETUP — Drive, paths, constants	Yes
2-4	Synthetic generators + save + verify	Yes (overwrites)
5-6	Trace URLs + PICK	Yes
7	Converter function	Yes
8-10	Download, convert, characterize	Yes (skips existing)
11-14	Trace loader + verify + load all	Yes
15+	Simulator (Stage 3 onward)	—

Add this guard at the top of any cell depending on setup:

```
assert 'OUT' in globals(), 'Run the SETUP cell first'
```

It fails with a clear message instead of a confusing `NameError` twenty lines down.

Part 6: Definition of done

Stages 1 and 2 are complete when all of these hold:

- [] 6 synthetic CSVs in `{OUT}/`, each visually matching its intended pattern

- [] `sequential` reports **exactly 1 unique delta**
- [] 4 real trace CSVs in `{OUT}/`, each with `skipped` near 0
- [] `trace_characteristics.csv` saved
- [] Clear `top5_share` gap between regular (bwaves, leslie3d) and irregular (bfs, pr) traces
- [] `Trace` class loads any trace by name
- [] All four assertions in Cell 13 pass
- [] Entire notebook survives `Runtime -> Run all` from scratch

Once all boxes are ticked, you have a clean data pipeline and can start Stage 3.

STAGE 3: The Prediction Model

What this stage is and isn't

Stage 3 builds and evaluates the **predictor** — the component that answers "given where we are now, what page comes next, and how sure am I?"

The important thing to understand: **this stage needs no memory engine at all**. You can measure prediction quality with zero simulated RAM, by simply asking "what did the model predict, and what actually came next?" This isolates predictor quality from paging effects, which makes debugging far easier and gives you a complete set of results before you build anything else.

Page faults come later (Stage 4). Right now you are only measuring whether the model can see the future.

Part 7: Why a Markov chain

The idea in one paragraph

Keep a notebook. Every time page A is followed by page B, write down "A → B" and add a tally mark. Over time some transitions accumulate many marks and others few. To predict what comes after A, look up all transitions starting from A and turn the tallies into percentages. That's a **probability distribution over next-page candidates**.

That's the entire algorithm. It's called a Markov chain, and it's the standard entry-level approach for this problem.

"Training" means counting, not learning

A Markov chain does **not** train like a neural network. No loss function, no gradient descent, no epochs. Training is literally walking the sequence and incrementing counters.

This has an important consequence: **the model can keep learning during evaluation**. Unlike a neural net that gets frozen after training, the Markov table keeps updating as new accesses stream past. This is what lets it adapt to phase changes.

State this explicitly in your writeup — "the model learns online" is a methodological point a reviewer will ask about.

What "order" means

- **Order-1**: context is the last 1 page. "After page 47, what comes next?"
- **Order-2**: context is the last 2 pages. "After the sequence (12, 47), what comes next?"

Order-2 is **more specific**, so it matches less often but is more reliable when it does match. Your results confirm this: order-2 has lower coverage and higher accuracy than order-1 on almost every trace.

Part 8: Keeping it lightweight

The predictor runs on **every memory access** — millions of times. If prediction costs more CPU or memory than the faults it prevents, the project is a net loss. Four techniques keep it cheap:

1. Bounded table (LRU eviction)

A naive Markov table stores a row for every page ever seen. On a real trace that explodes. Instead, cap the number of entries and evict the least-recently-used one when full.

Python's `OrderedDict` gives you this in three operations: - `move_to_end(key)` — mark as recently used - `popitem(last=False)` — evict the oldest - normal dict lookup — $O(1)$

2. Top-K successors

Don't store every page that ever followed page A. Store the top 2-4. If the correct answer isn't in your top few candidates, confidence was too low to act on anyway.

3. Integer counters, not floats

Store tally counts as small integers. Probabilities are computed **only when you ask for a prediction** — the hot path (update) never does division.

4. Periodic decay

Halve all counters every N updates using a bit-shift (`count >>= 1`). This makes old behaviour fade automatically, so the model tracks the *current* program phase rather than being anchored to startup behaviour. It also keeps counters small.

Part 9: The code

Cell 1 — The predictors

```
#cell 1 – predictors
from collections import OrderedDict

class MarkovPredictor:
    """Counts observed transitions. Bounded LRU table, top-K successors."""

    def __init__(self, capacity=256, max_succ=2, order=2,
                 decay_every=25_000, min_count=1):
        self.capacity = capacity      # max distinct contexts remembered
        self.max_succ = max_succ      # successors kept per context
        self.order = order            # how many past pages form the context
        self.decay_every = decay_every # halve counters every N updates
        self.min_count = min_count
        self.table = OrderedDict()    # context -> {successor: count}
        self.n_updates = 0
        self.evictions = 0            # diagnostic: capacity pressure

    def _key(self, history):
        """Context = last `order` pages. None if not enough history yet."""
        if len(history) < self.order:
            return None
        return tuple(history[-self.order:])
```

```

def update(self, history, actual_next):
    """Record that `actual_next` followed this context."""
    k = self._key(history)
    if k is None:
        return

    if k in self.table:
        self.table.move_to_end(k)          # mark recently used
    else:
        if len(self.table) >= self.capacity:
            self.table.popitem(last=False) # evict least-recently-used
            self.evictions += 1
        self.table[k] = {}

    succ = self.table[k]
    succ[actual_next] = succ.get(actual_next, 0) + 1

    if len(succ) > self.max_succ:          # keep only top-K
        del succ[min(succ, key=succ.get)]

    self.n_updates += 1
    if self.n_updates % self.decay_every == 0:
        self._decay()

def _decay(self):
    """Halve all counts so old behaviour fades."""
    for succ in self.table.values():
        for s in list(succ):
            succ[s] >>= 1
            if succ[s] == 0:
                del succ[s]

def predict(self, history, top_n=1):
    """Return [(page, confidence), ...] sorted by confidence."""
    k = self._key(history)
    if k is None or k not in self.table:
        return []
    succ = self.table[k]
    total = sum(succ.values())
    if total == 0:
        return []
    ranked = sorted(succ.items(), key=lambda kv: -kv[1])[:top_n]
    return [(p, c / total) for p, c in ranked if c >= self.min_count]

def stats(self):
    return {'entries': len(self.table),
            'evictions': self.evictions,
            'fill': round(len(self.table) / self.capacity, 3)}

class StridePredictor:
    """Predicts next = last + stride when recent deltas agree."""

    def __init__(self, confirm=3):
        self.confirm = confirm          # matching deltas required before firing

    def predict(self, history, top_n=1):
        if len(history) < self.confirm + 1:
            return []
        d = [history[i] - history[i - 1] for i in range(-self.confirm, 0)]
        if len(set(d)) != 1 or d[0] == 0:
            return []
        return [(history[-1] + d[0], 1.0)]

    def update(self, history, actual_next):
        pass                            # stateless – nothing to learn

    def stats(self):
        return {}

```

```

class HybridPredictor:
    """Order-2 Markov first (precise), order-1 fallback (broad), then stride."""

    def __init__(self, capacity=256, max_succ=2, decay_every=25_000):
        self.m2 = MarkovPredictor(capacity, max_succ, order=2,
                                   decay_every=decay_every)
        self.m1 = MarkovPredictor(capacity, max_succ, order=1,
                                   decay_every=decay_every)
        self.st = StridePredictor()
        self.source_counts = {'m2': 0, 'm1': 0, 'stride': 0, 'none': 0}

    def predict(self, history, top_n=1):
        for tag, p in (('m2', self.m2), ('m1', self.m1), ('stride', self.st)):
            r = p.predict(history, top_n)
            if r:
                self.source_counts[tag] += 1
                return r
        self.source_counts['none'] += 1
        return []

    def update(self, history, actual_next):
        self.m2.update(history, actual_next)
        self.m1.update(history, actual_next)

    def stats(self):
        return {'sources': dict(self.source_counts), **self.m2.stats()}

print('predictors ready')

```

Line-by-line explanation of the tricky parts

OrderedDict + move_to_end + popitem(last=False) This is an LRU cache in three lines. Without it, a real trace with hundreds of thousands of distinct pages would blow up memory — which would contradict the lightweight claim.

del succ[min(succ, key=succ.get)] Finds the successor with the lowest count and deletes it. This enforces the top-K cap.

succ[s] >>= 1 Integer halving via bit-shift, not float division. Counts stay small integers.

return [(p, c / total) for p, c in ranked ...] Division happens **only here**, in `predict()`. The hot path (`update()`) never divides.

The HybridPredictor fallback chain Tries order-2 first (most specific). If it has no match, falls back to order-1 (broader). If that fails too, tries stride (cheapest, works on arithmetic patterns the table is too small to hold). `source_counts` records which one fired — this becomes direct evidence for the "workload-aware" claim.

Cell 2 — The evaluator

```

#cell 2 – prediction quality evaluator
def eval_predictor(trace, predictor, warmup_frac=0.25, hist_len=8):
    """Stream the trace; learn throughout, measure only past warmup."""
    pages = trace.pages
    split = int(len(pages) * warmup_frac)
    history = []
    attempted = correct = no_pred = 0
    conf_sum = 0.0

```

```

for i in range(len(pages) - 1):
    history.append(int(pages[i]))
    if len(history) > hist_len:
        history.pop(0)
    actual = int(pages[i + 1])

    if i >= split:                                # measure only past warmup
        preds = predictor.predict(history, top_n=1)
        if not preds:
            no_pred += 1
        else:
            page, conf = preds[0]
            attempted += 1
            conf_sum += conf
            if page == actual:
                correct += 1

    predictor.update(history, actual)              # learns throughout

n_eval = max(1, len(pages) - 1 - split)
coverage = attempted / n_eval
accuracy = correct / attempted if attempted else 0.0
return {
    'coverage': coverage,
    'accuracy': accuracy,
    'useful': coverage * accuracy,                # combined score
    'mean_conf': conf_sum / attempted if attempted else 0.0,
    'extra': predictor.stats(),
}

```

The three metrics — and why you need all three

Metric	Definition	What it captures
Coverage	attempted ÷ total	How often the predictor was <i>willing</i> to guess
Accuracy	correct ÷ attempted	Of the guesses made, how many were right
Useful	coverage × accuracy	Fraction of all accesses where it both spoke <i>and</i> was right

These trade off. A predictor that only guesses when certain has **high accuracy, low coverage**. One that always guesses has the reverse. Neither number alone tells the story.

useful is the headline — it's what a prefetcher actually delivers. But report all three, because `useful` alone hides *why* a predictor failed: did it stay silent, or did it guess wrong? Those call for different fixes.

Note `predictor.update()` **sits outside the `if i >= split` block**. The model learns from every access, including the evaluation portion. That's deliberate — it's online learning, and it's what makes adaptation possible.

Cell 3 — Rebuild TRACES

```

#cell 3 – rebuild TRACES as Trace objects
import os, pandas as pd, numpy as np, matplotlib.pyplot as plt

SYNTHETIC = ['sequential', 'strided', 'cyclic', 'clustered', 'random', 'phase_changing']
REAL = PICK
ALL_TRACES = [n for n in SYNTHETIC + REAL if os.path.exists(f'{OUT}/{n}.csv')]

TRACES = {n: load_trace(n, limit=200_000) for n in ALL_TRACES}

```

```
assert all(hasattr(t, 'pages') for t in TRACES.values()), \
    'TRACES holds raw lists – check for a stray "TRACES =" assignment'
print(f'{len(TRACES)} Trace objects loaded')
for t in TRACES.values():
    print(' ', t)
```

Why this cell exists: a very common bug is accidentally assigning a dict of *raw lists* to `TRACES` (e.g. from the synthetic generation cell) instead of `Trace` objects. Everything downstream then fails with `'list' object has no attribute 'pages'` deep in the call stack.

The assertion catches it immediately with a clear message. Add the same assertion to any cell that consumes traces.

Cell 4 — Freeze the configuration

```
#cell 4 – FROZEN HYPERPARAMETERS (single source of truth)
FROZEN = dict(capacity=256, max_succ=2, order=2, decay_every=25_000)

# Justification (from sweeps in cells 8 and below):
# capacity=256 -> useful_real is FLAT (0.783) from 256 to 4096; 16x the
#               memory buys nothing on real traces. At 256: 8.5 KB table,
#               ~2 pages, 1.66% overhead at 128 frames.
# max_succ=2    -> 2/4/8 differ by <0.0002; pick the cheapest
# order=2       -> beats order-1 by ~4.5 points (a real effect, not noise)
# decay=25000   -> marginally best, negligible cost

def make_predictor(kind):
    if kind == 'stride': return StridePredictor()
    if kind == 'markov-o1': return MarkovPredictor(**{**FROZEN, 'order': 1})
    if kind == 'markov-o2': return MarkovPredictor(**FROZEN)
    if kind == 'hybrid': return HybridPredictor(
        capacity=FROZEN['capacity'],
        max_succ=FROZEN['max_succ'],
        decay_every=FROZEN['decay_every'])
    raise ValueError(kind)

KINDS = ['stride', 'markov-o1', 'markov-o2', 'hybrid']

import json
with open(f'{BASE}/frozen_config.json', 'w') as f:
    json.dump(FROZEN, f, indent=2)
print('FROZEN:', FROZEN)
```

Why freezing matters

Once you pick hyperparameters, **write them down and never change them again**. Re-tuning after seeing results you dislike is the single most common way to invalidate an evaluation.

`make_predictor()` exists so there is exactly **one** place that constructs predictors. If a stray `CAP = 16384` lives in another cell, some results use one config and some another, and the numbers stop being comparable.

Why not just take the best sweep result?

The accuracy sweep will always favour the **largest** capacity, because more memory never hurts accuracy. But the top several rows typically differ by ~ 0.0003 , which is noise. Choosing the smallest configuration that performs equally well is the correct decision once memory cost is part of the picture — and it makes the lightweight claim far stronger.

Keep the justification comments. When you write the paper, that block is your hyperparameter-selection methodology, already drafted.

Cell 5 — Main results table

```
#cell 5 – coverage / accuracy / useful across all traces
assert all(hasattr(t, 'pages') for t in TRACES.values()), 're-run cell 3'

rows = []
for name, t in TRACES.items():
    kind_label = 'real' if name in REAL else 'synthetic'
    for k in KINDS:
        s = eval_predictor(t, make_predictor(k))
        rows.append({'trace': name, 'type': kind_label, 'predictor': k,
                    'coverage': round(s['coverage'], 3),
                    'accuracy': round(s['accuracy'], 3),
                    'useful': round(s['useful'], 3)})

results = pd.DataFrame(rows)
results.to_csv(f'{BASE}/predictor_results.csv', index=False)

for metric in ['coverage', 'accuracy', 'useful']:
    print(f'\n===== {metric.upper()} =====')
    display(results.pivot(index='trace', columns='predictor', values=metric)[KINDS])

print('\n===== USEFUL (heatmap) =====')
display(results.pivot(index='trace', columns='predictor', values='useful')[KINDS]
        .style.background_gradient(cmap='RdYlGn', axis=None).format('{:.3f}'))
```

What the results showed

Trace	stride	markov-o1	markov-o2	hybrid
410.bwaves-s0	0.256	0.788	0.761	0.804
437.leslie3d-s0	0.002	0.843	0.816	0.844
bfs-3	0.000	0.669	0.673	0.706
pr-3	0.000	0.458	0.881	0.892
clustered	0.000	0.097	0.081	0.099
cyclic	0.800	1.000	1.000	1.000
random	0.000	0.000	0.000	0.000
sequential	1.000	0.000	0.000	1.000
strided	0.997	0.000	0.000	0.997

Four findings worth stating in the paper:

1. **The hybrid wins or ties everywhere.** It is never the weakest predictor on any trace, because it falls back to whichever mechanism suits the current workload.
2. **Stride prefetching is nearly worthless on real LLC traces** (0.256, 0.002, 0.000, 0.000). This motivates the Markov approach directly — and it's a consequence of the LLC filtering discussed in Stage 1: the clean strides were already absorbed by L1/L2.

3. **clustered at ~0.10 across all predictors is your single most important row.** Every predictor confidently guesses and is ~90% wrong. Without a cost-aware decision engine, a system here would waste 90% of its prefetch I/O *and* evict useful pages. **This row is the empirical justification for cost-awareness** — data, not assertion.
4. **random at exactly 0.000 everywhere is correct behaviour.** The predictors degrade to silence rather than noise.

On sequential/strided scoring 0.000 for Markov: this is not a failure. Those synthetic traces have a 10,000-page working set, far larger than the 256-entry table. The hybrid rescues them via the stride fallback — which is precisely the design intent. Explain this in the writeup rather than letting the zeros look like bugs.

Cell 6 — Source attribution

```
#cell 6 — which sub-predictor fired
attrib = []
for name, t in TRACES.items():
    h = make_predictor('hybrid')
    eval_predictor(t, h)
    src = h.stats()['sources']
    tot = max(1, sum(src.values()))
    print(f"{name:18s} " + " ".join(f"{k}={v/tot:.0%}" for k, v in src.items()))
    attrib.append({'trace': name, **{k: round(v/tot, 3) for k, v in src.items()}})

att = pd.DataFrame(attrib).set_index('trace')
att.to_csv(f'{BASE}/predictor_attribution.csv')
display(att)

att[['m2', 'm1', 'stride', 'none']].plot(
    kind='barh', stacked=True, figsize=(9,5),
    color=['#2a9d8f', '#8ab17d', '#e9c46a', '#e76f51'])
plt.xlabel('fraction of predictions'); plt.title('Which sub-predictor fired')
plt.legend(loc='center left', bbox_to_anchor=(1,0.5))
plt.tight_layout(); plt.show()
```

What this proves

This is the **direct evidence for "workload-aware"** in your project title. Observed at capacity=256:

Trace	m2	m1	stride	none
sequential	0%	0%	100%	0%
strided	0%	0%	100%	0%
cyclic	100%	0%	0%	0%
clustered	80%	18%	0%	2%
random	0%	3%	0%	97%
410.bwaves-s0	91%	8%	1%	1%
437.leslie3d-s0	93%	6%	0%	2%
bfs-3	85%	11%	0%	4%
pr-3	93%	4%	0%	3%

Different workloads engage genuinely different mechanisms — shown, not claimed. Two rows deserve comment:

- **sequential/strided = 100% stride.** The Markov table is too small for their working sets, so the cheap stride predictor covers them entirely. The fallback chain is doing real work.
- **random = 97% "none".** The predictor correctly stays silent on unpredictable data. That self-awareness is valuable and worth calling out.

Cell 7 — Memory footprint

```
#cell 7 - memory footprint (information-theoretic, NOT Python overhead)
CTX_BYTES = FROZEN['order'] * 8      # 64-bit page numbers
SUCC_BYTES = 9                      # 8B page + 1B saturating counter

MAX_BYTES = (FROZEN['capacity'] * CTX_BYTES
              + FROZEN['capacity'] * FROZEN['max_succ'] * SUCC_BYTES)

foot = []
for name, t in TRACES.items():
    m = MarkovPredictor(**FROZEN)
    eval_predictor(t, m)
    n_ctx = len(m.table)
    n_succ = sum(len(s) for s in m.table.values())
    raw_b = n_ctx*CTX_BYTES + n_succ*SUCC_BYTES
    foot.append({'trace': name, 'contexts': n_ctx, 'successors': n_succ,
                'succ_per_ctx': round(n_succ/n_ctx, 2) if n_ctx else 0,
                'evictions': m.evictions,
                'table_KB': round(raw_b/1024, 1),
                'equiv_4K_pages': round(raw_b/4096, 1),
                'pct_of_max': round(100*raw_b/MAX_BYTES, 1)})

fp = pd.DataFrame(foot)
display(fp)
fp.to_csv(f'{BASE}/memory_footprint.csv', index=False)

print(f"\nconfig: capacity={FROZEN['capacity']}, order={FROZEN['order']}, "
      f"max_succ={FROZEN['max_succ']}")
print(f"bounded ceiling : {MAX_BYTES/1024:.1f} KB (~{MAX_BYTES/4096:.1f} pages)")
print(f"observed max      : {fp.table_KB.max():.1f} KB")
print(f"observed mean     : {fp.table_KB.mean():.1f} KB")

print('\noverhead as % of simulated RAM:')
print(f"{'frames':>8} {'RAM_KB':>9} {'overhead_%':>11}")
for frames in [128, 256, 512, 1024]:
    print(f"{'frames':>8} {'frames*4':>9} {100*MAX_BYTES/1024/(frames*4):>10.2f}%")
```

Why NOT to use `sys.getsizeof()`

This is an important methodological point. `sys.getsizeof()` on a Python integer returns **28 bytes**, because of CPython object overhead. But the *information* being stored is a page number and a small count — 8 bytes and 1 byte in a real kernel or hardware implementation.

Measuring with `sys.getsizeof()` gives ~1156 KB (≈289 pages), which would make the predictor cost **more memory than the RAM being simulated** and destroy the lightweight claim entirely. That number measures Python's interpreter, not your algorithm.

The information-theoretic estimate is the honest one: - context key: `order × 8` bytes (64-bit page numbers) - successor: 8 bytes page + 1 byte saturating counter = 9 bytes

The bounded ceiling

At `capacity=256, order=2, max_succ=2`: - contexts: $256 \times 16 = 4,096$ bytes - successors: $512 \times 9 = 4,608$ bytes - **total = 8.5 KB $\approx$ 2.1 pages**

The word **bounded** matters. This isn't an average that might blow up on some unseen workload — the LRU eviction and top-K cap make it a **hard ceiling by construction**, regardless of trace.

Observed overhead

frames	RAM (KB)	overhead
128	512	1.66%
256	1024	0.83%
512	2048	0.42%
1024	4096	0.21%

Under 2% at every realistic memory size. That's a defensible lightweight claim.

Note on `succ_per_ctx`: `clustered` is the only trace near 2.0 (1.95), meaning its contexts genuinely have multiple plausible successors. That's *why* its accuracy is 10%. The memory table and the accuracy table are telling the same story from different angles — clustered workloads are inherently ambiguous, which is exactly the case the cost-aware engine must handle by declining to prefetch.

Cell 8 — Capacity sweep

```
#cell 8 – capacity vs accuracy vs overhead
rows = []
for cap in [128, 256, 512, 1024, 2048, 4096]:
    cfg = {'**FROZEN', 'capacity': cap}
    max_b = cap*CTX_BYTES + cap*cfg['max_succ']*SUCC_BYTES
    real_s, all_s = [], []
    for name, t in TRACES.items():
        u = eval_predictor(t, MarkovPredictor(**cfg))['useful']
        all_s.append(u)
        if name in REAL:
            real_s.append(u)
    rows.append({'capacity': cap, 'table_KB': round(max_b/1024,1),
                'pages': round(max_b/4096,1),
                'useful_real': round(np.mean(real_s),3),
                'useful_all': round(np.mean(all_s),3),
                'ovh_128f': f"{100*max_b/1024/512:.1f}%",
                'ovh_512f': f"{100*max_b/1024/2048:.1f}%"})

sw = pd.DataFrame(rows)
display(sw)
sw.to_csv(f'{BASE}/capacity_sweep.csv', index=False)

fig, ax1 = plt.subplots(figsize=(8,4.5))
ax1.plot(sw.capacity, sw.useful_real, 'o-', color='#2a9d8f', label='useful (real)')
ax1.plot(sw.capacity, sw.useful_all, 's--', color='#8ab17d', label='useful (all)')
ax1.set_xscale('log', base=2); ax1.set_xlabel('table capacity')
ax1.set_ylabel('useful prediction rate'); ax1.set_ylim(0,1)
ax2 = ax1.twinx()
ax2.plot(sw.capacity, sw.table_KB, '^-', color='#e76f51', label='table KB')
ax2.set_ylabel('table size (KB)')
ax1.legend(loc='lower right'); ax2.legend(loc='center right')
```

```
plt.title('Accuracy is flat; memory is not'); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
```

The result that justifies capacity=256

capacity	table_KB	useful_real	useful_all	ovh @128f
128	4.2	0.783	0.466	0.8%
256	8.5	0.783	0.466	1.7%
512	17.0	0.783	0.466	3.3%
1024	34.0	0.783	0.466	6.6%
2048	68.0	0.783	0.566	13.3%
4096	136.0	0.783	0.566	26.6%

`useful_real` is **completely flat at 0.783** across a 32× memory range. You would be paying 32× the memory for exactly zero benefit on real workloads.

The dual-axis chart makes this argument visually: one line dead flat, the other climbing steeply. It is paper-ready as-is.

Honest caveat to state in the writeup: `useful_all` *does* drop below capacity 2048, entirely because the synthetic `sequential` and `strided` traces need larger tables. The configuration is tuned for **real** workloads; synthetic traces with artificially large working sets are covered by the stride fallback instead. The attribution chart (Cell 6) is the evidence that this works.

Cell 9 — Phase-change adaptation

```
#cell 9 – adaptation across workload phases
def rolling_accuracy(trace, predictor, window=300, warmup_frac=0.0,
                    hist_len=8, step=50):
    pages = trace.pages
    split = int(len(pages) * warmup_frac)
    hist, hits, xs, ys = [], [], [], []
    for i in range(len(pages) - 1):
        hist.append(int(pages[i]))
        if len(hist) > hist_len:
            hist.pop(0)
        actual = int(pages[i + 1])
        if i >= split:
            p = predictor.predict(hist, top_n=1)
            hits.append(1 if (p and p[0][0] == actual) else 0)
            if len(hits) >= window and len(hits) % step == 0:
                xs.append(i)
                ys.append(np.mean(hits[-window:]))
        predictor.update(hist, actual)
    return xs, ys

t = TRACES['phase_changing']
n = len(t.pages)
b1, b2 = n // 3, 2 * n // 3

# ---- per-predictor panels (avoids overplotting) ----
fig, axes = plt.subplots(len(KINDS), 1, figsize=(12, 2.2 * len(KINDS)),
                        sharex=True, sharey=True)
for ax, k in zip(axes, KINDS):
    xs, ys = rolling_accuracy(t, make_predictor(k))
    ax.plot(xs, ys, lw=1.0, color='#2a9d8f')
    ax.axvline(b1, ls='--', c='gray', alpha=.6)
```

```

    ax.axvline(b2, ls='--', c='gray', alpha=.6)
    ax.set_ylim(-.05, 1.1)
    ax.set_ylabel(k, fontsize=9)
    ax.grid(alpha=.3)

axes[0].set_title('Adaptation across phases', pad=28)
axes[0].text(b1/2, 1.22, 'sequential', ha='center', fontsize=9)
axes[0].text((b1+b2)/2, 1.22, 'cyclic', ha='center', fontsize=9)
axes[0].text((b2+n)/2, 1.22, 'random', ha='center', fontsize=9)
axes[-1].set_xlabel('access #')
plt.tight_layout()
plt.savefig(f'{BASE}/phase_adaptation.png', dpi=150, bbox_inches='tight')
plt.show()

# ---- zoom: how fast does each predictor recover after a phase change? ----
fig, axes = plt.subplots(1, 2, figsize=(13, 4), sharey=True)
for ax, (b, label) in zip(axes, [(b1, 'seq -> cyclic'), (b2, 'cyclic -> random')]):
    for k in KINDS:
        xs, ys = rolling_accuracy(t, make_predictor(k), window=50, step=10)
        pts = [(x, y) for x, y in zip(xs, ys) if b - 300 < x < b + 2500]
        if pts:
            ax.plot([p[0] for p in pts], [p[1] for p in pts], lw=1.1, label=k)
    ax.axvline(b, ls='--', c='gray')
    ax.set_title(f'transition: {label}')
    ax.set_xlabel('access #')
    ax.grid(alpha=.3)
axes[0].set_ylabel('rolling accuracy (window=50)')
axes[0].legend(fontsize=8)
plt.tight_layout()
plt.savefig(f'{BASE}/phase_recovery_zoom.png', dpi=150, bbox_inches='tight')
plt.show()

# ---- quantify recovery time ----
def recovery_time(trace, predictor, boundary, window=50, step=10,
                  target=0.9, lookahead=5000):
    xs, ys = rolling_accuracy(trace, predictor, window=window, step=step)
    post = [(x, y) for x, y in zip(xs, ys) if boundary <= x <= boundary + lookahead]
    if not post:
        return None
    plateau = max(y for _, y in post)
    if plateau <= 0:
        return 0
    for x, y in post:
        if y >= target * plateau:
            return x - boundary
    return None

print('accesses to reach 90% of post-transition plateau:')
print(f'{"predictor":<12}{"seq->cyclic":>14}{"cyclic->random":>17}')
for k in KINDS:
    r1 = recovery_time(t, make_predictor(k), b1)
    r2 = recovery_time(t, make_predictor(k), b2)
    f = lambda r: 'n/a' if r is None else str(r)
    print(f'{k:<12}{f(r1):>14}{f(r2):>17}')

```

Why separate panels instead of one overlaid plot

Plotting all four predictors on one axis **overplots** — the last line drawn (hybrid) hides the others, and the apparent story becomes "everything is perfect," which is an artifact. Separate panels let each predictor's behaviour be read independently.

Why `warmup_frac=0.0` here

Unlike `eval_predictor`, this function starts measuring from access 0. That includes cold-start behaviour, which is interesting in itself: how many accesses does each predictor need before it becomes useful?

Why `window=300` not 2000

A wide rolling window **smears** across phase transitions, producing flat plateaus with vertical jumps that hide the actual dynamics. 300 resolves the transitions; the zoom plots use 50 for finer detail.

What the figure shows

- **stride** — perfect in sequential, drops to ~ 0.84 in cyclic (the loop contains stride-like runs), dies at random
- **markov-o1 / markov-o2** — zero in sequential (256-entry table vs 10,000-page working set), perfect in cyclic, zero in random
- **hybrid** — holds **1.0 across both productive phases**

That last panel is the result. **The hybrid is the only predictor that is never the weakest**, because it switches which mechanism it relies on as the workload changes. That is the adaptivity claim, demonstrated rather than asserted.

The `cyclic → random` recovery column will likely show 0 or `n/a` for everything, because the plateau there is zero by construction. That's expected, not a bug.

Cell 10 — Save summary

```
#cell 10 – save summary
summary = {
    'frozen_config': FROZEN,
    'n_traces':      len(TRACES),
    'real_traces':   REAL,
    'synthetic':     SYNTHETIC,
    'table_KB':      round(MAX_BYTES/1024, 1),
    'overhead_128f': f"{100*MAX_BYTES/1024/512:.2f}%",
    'hybrid_mean_useful_real': float(
        results[(results.predictor=='hybrid') & (results.type=='real')].useful.mean()),
    'hybrid_mean_useful_all': float(
        results[results.predictor=='hybrid'].useful.mean()),
}
with open(f'{BASE}/stage3_summary.json','w') as f:
    json.dump(summary, f, indent=2, default=str)
print(json.dumps(summary, indent=2, default=str))
!ls -la {BASE}/*.csv {BASE}/*.json
```

Part 10: Stage 3 definition of done

- [] All three predictor classes defined without error
- [] `TRACES` holds `Trace` objects (assertion passes)
- [] `FROZEN` defined in exactly one cell; no stray `CAP` variables anywhere
- [] Coverage / accuracy / useful tables produced for all traces × all predictors
- [] Hybrid wins or ties on every real trace

- [] Attribution chart shows different mechanisms firing on different workloads
- [] Memory footprint under 2% overhead at 128 frames
- [] Capacity sweep shows `useful_real` flat across the swept range
- [] Phase-adaptation figure produced with non-overlapping labels
- [] `stage3_summary.json` saved

Part 11: Artifacts produced

Files written to `{BASE}` that feed directly into the paper:

File	Goes into
<code>frozen_config.json</code>	Methodology — hyperparameter selection
<code>predictor_results.csv</code>	Results — main comparison table
<code>predictor_attribution.csv</code>	Results — workload-awareness evidence
<code>memory_footprint.csv</code>	Results — overhead analysis
<code>capacity_sweep.csv</code>	Results — design-space exploration
<code>phase_adaptation.png</code>	Results — adaptivity figure
<code>phase_recovery_zoom.png</code>	Results — transition dynamics
<code>stage3_summary.json</code>	Reproducibility record

Part 12: What Stage 3 does NOT tell you

An important limitation to keep in mind before moving on.

Prediction accuracy is **not** the same as performance benefit. A correct prediction can still be harmful: if RAM is full, prefetching the predicted page evicts something else — and if that evicted page was about to be reused, the prefetch **caused** a fault instead of preventing one.

This is exactly why the `clustered` result (10% accuracy, 98% coverage) matters so much. A naive system acting on every prediction there would be actively worse than doing nothing.

Turning "80% prediction accuracy" into "X% fewer page faults" requires the **Memory Engine** — page table, frame table, eviction policy, and the cost-aware decision layer that decides whether a prediction is worth acting on. That is Stage 4.

Part 13: Notebook structure (all stages)

Organise so `Runtime -> Run all` works from scratch after any reconnect:

Section	Cells	Idempotent?
Stage 1	SETUP (Drive, paths, constants)	Yes
	Synthetic generators + save + verify	Yes (overwrites)
	Trace URLs + PICK	Yes
	Converter function	Yes
	Download, convert, characterise	Yes (skips existing)

Section	Cells	Idempotent?
Stage 2	Trace class + load_trace	Yes
	Verification assertions	Yes
Stage 3	Predictor classes	Yes
	eval_predictor	Yes
	Rebuild TRACES	Yes
	FROZEN + make_predictor	Yes
	Results, attribution, footprint, sweep, phases	Yes
Stage 4	MemoryEngine + policies	Yes
	Baseline sweep + verification	Yes
Stage 5	Diagnostic (14b)	Yes
	Decision engines (15, 15b)	Yes
	Simulator with shadow (16)	Yes
	Ablation + summary (17)	Yes
Stage 6	Figures, robustness, sensitivity (next)	—

STAGE 4: The Memory Engine

What this stage does

Stage 3 answered "*can we predict the next page?*" — the answer was yes, roughly 78% useful accuracy on real traces.

Stage 4 answers a different question: **"how many page faults actually happen?"**

This is the stage that turns abstract prediction accuracy into the number that matters. And crucially, it is built and verified **with no prediction at all**. You implement the engine, implement the four classical replacement policies, and prove they behave correctly. Only then (Stage 5) does the predictive layer get added on top.

Why this order is non-negotiable: if the baselines are wrong, every prefetching result you produce afterwards is meaningless — and you will not know it, because a buggy baseline still produces plausible-looking numbers.

Part 14: Core concepts

What a page fault is, in the simulator

A real CPU has a page table where each entry has a "present" bit. If the bit is 0, hardware raises an interrupt and the OS fetches the page from disk.

In your simulator this is one lookup: **is this page ID in my resident set?** If yes, hit. If no, that's a fault — increment the counter, load the page, evict something if RAM is full.

Detection is trivial. All the interesting engineering is in what happens *around* the lookup.

Why fault rate is the metric

A page fault stalls execution while data is fetched from disk. Disk is roughly 100,000x slower than RAM, so each fault is enormously expensive.

Lower fault rate = faster program. This is the opposite direction from Stage 3's metrics:

Metric	Direction	Why
coverage, accuracy, useful	Higher is better	They count successes
fault_rate, wasted prefetches, evictions	Lower is better	They count problems

Why you can never reach zero

Compulsory faults are unavoidable: the first time a page is ever touched, the data genuinely is not in RAM and nothing can predict it out of nowhere. OPT's fault count is effectively the floor set by those compulsory faults, which is exactly why OPT is the benchmark rather than zero.

The four policies

Policy	Rule	Notes
FIFO	Evict whichever page arrived first	Simple; ignores behaviour entirely
LRU	Evict the page untouched for longest	Uses recency history
Clock	Second-chance approximation of LRU	Cheap to implement in hardware
OPT	Evict whichever page is needed furthest in the future	Impossible in practice — requires knowing the future

OPT (Belady's algorithm) is *allowed to cheat* because it is defined as the theoretical lower bound. It exists to tell you how much headroom a practical policy is leaving on the table.

Part 15: The code

Cell 11 — The Memory Engine

```
#cell 11 – policy-agnostic memory engine
from collections import OrderedDict, deque

class MemoryEngine:
    """Page table + frame table + fault handler. Knows nothing about policy."""

    def __init__(self, n_frames, policy):
        self.n_frames = n_frames
        self.policy = policy          # pluggable eviction policy
        self.resident = set()        # pages currently in RAM
        self.reset_stats()

    def reset_stats(self):
        self.n_access = 0
        self.n_fault = 0
        self.n_evict = 0
        # prefetch bookkeeping (unused by baselines, needed in Stage 5)
        self.prefetched = {}         # page -> still-unused flag
        self.n_prefetch = 0
        self.n_pf_used = 0
        self.n_pf_wasted = 0

    # ---- the ONLY way a page enters RAM ----
    def _install(self, page, i, is_prefetch=False):
        if page in self.resident:
            return
        if len(self.resident) >= self.n_frames:
            victim = self.policy.choose_victim(self.resident, i)
            self.resident.discard(victim)
            self.policy.on_evict(victim)
            self.n_evict += 1
            if self.prefetched.pop(victim, False):
                self.n_pf_wasted += 1      # prefetched but never used
        self.resident.add(page)
        self.policy.on_insert(page, i)
        if is_prefetch:
            self.prefetched[page] = True
            self.n_prefetch += 1

    def access(self, page, i):
        """Demand access. Returns True if it was a fault."""
        self.n_access += 1
        hit = page in self.resident
```

```

    if self.prefetched.pop(page, False):
        self.n_pf_used += 1          # a prefetch paid off

    if hit:
        self.policy.on_hit(page, i)
        return False

    self.n_fault += 1
    self._install(page, i, is_prefetch=False)
    return True

def prefetch(self, page, i):
    """Speculative load. Never bypasses the fault handler."""
    if page in self.resident:
        return False
    self._install(page, i, is_prefetch=True)
    return True

def stats(self):
    return {
        'accesses':    self.n_access,
        'faults':      self.n_fault,
        'fault_rate':  self.n_fault / max(1, self.n_access),
        'evictions':   self.n_evict,
        'prefetches':  self.n_prefetch,
        'pf_used':     self.n_pf_used,
        'pf_wasted':   self.n_pf_wasted,
        'pf_precision': self.n_pf_used / max(1, self.n_prefetch),
    }

print('MemoryEngine ready')

```

The architectural rule, enforced structurally

The prediction/decision path must never alter correctness — only performance.

A completely broken predictor should make the system slow, never wrong.

This is not a comment or a convention here — it is enforced by the code's shape. `_install()` is **the only path into RAM**. Both `access()` and `prefetch()` go through it, and both evict via the normal policy. Prefetching can never bypass the fault handler or install a page by some special route.

The practical payoff: you can disable the entire predictive layer and the simulator still runs correctly as plain LRU. That makes debugging vastly easier, because any incorrect behaviour is provably in the engine, not in the prediction.

Why the engine is policy-agnostic

The engine owns the page table, frame table, and fault handler. It knows nothing about *which* page to evict — it calls `policy.choose_victim()` and accepts the answer.

This means FIFO, LRU, Clock, and OPT all run through **identical machinery**. Any difference in results is caused purely by the policy, not by implementation quirks in four separate simulators. That is what makes the comparison fair.

The policy interface

Four methods, all optional no-ops if a policy doesn't need them:

Method	Called when
<code>on_insert(page, i)</code>	A page enters RAM
<code>on_hit(page, i)</code>	A resident page is accessed
<code>on_evict(page)</code>	A page is removed
<code>choose_victim(resident, i)</code>	RAM is full and something must go

The prefetch bookkeeping

Four counters exist in the engine but stay at zero for all of Stage 4:

- `n_prefetch` — speculative loads issued
- `n_pf_used` — prefetched pages later actually accessed (**the prefetch paid off**)
- `n_pf_wasted` — prefetched pages evicted before ever being used (**wasted I/O**)
- `pf_precision` — $\text{used} \div \text{issued}$

They're built in now so Stage 5 needs no changes to the engine. The `pf_wasted` counter in particular is what makes "useful prefetch rate" measurable — the metric that separates your work from papers that only report raw accuracy.

Cell 12 — The four policies

```
#cell 12 – eviction policies
class FIFO:
    def __init__(self, **kw): self.q = deque()
    def on_insert(self, page, i): self.q.append(page)
    def on_hit(self, page, i): pass
    def on_evict(self, page): pass
    def choose_victim(self, resident, i):
        while self.q:
            p = self.q.popleft()
            if p in resident:
                return p
        return next(iter(resident))

class LRU:
    def __init__(self, **kw): self.od = OrderedDict()
    def on_insert(self, page, i): self.od[page] = i; self.od.move_to_end(page)
    def on_hit(self, page, i): self.od[page] = i; self.od.move_to_end(page)
    def on_evict(self, page): self.od.pop(page, None)
    def choose_victim(self, resident, i):
        for p in self.od:
            if p in resident:
                return p
        return next(iter(resident))

class Clock:
    """Second-chance approximation of LRU."""
    def __init__(self, **kw):
        self.ring, self.ref, self.hand = [], {}, 0
    def on_insert(self, page, i):
        self.ring.append(page); self.ref[page] = 1
    def on_hit(self, page, i): self.ref[page] = 1
    def on_evict(self, page):
        if page in self.ring:
            idx = self.ring.index(page)
            self.ring.pop(idx); self.ref.pop(page, None)
            if self.hand > idx: self.hand -= 1
    def choose_victim(self, resident, i):
```

```

while True:
    if not self.ring: return next(iter(resident))
    self.hand %= len(self.ring)
    p = self.ring[self.hand]
    if p not in resident:
        self.ring.pop(self.hand); self.ref.pop(p, None); continue
    if self.ref.get(p, 0) == 0:
        return p
    self.ref[p] = 0
    self.hand += 1

print('policies ready')

```

How each works

FIFO — a simple queue. `popleft()` gives the oldest page. The `while` loop skips entries already evicted by other means, keeping the queue consistent with the resident set.

LRU — an `OrderedDict` where `move_to_end()` marks a page as recently used. The **first** item in iteration order is therefore the least recently used. Both `on_insert` and `on_hit` refresh it, which is what makes LRU recency-aware.

Clock — pages sit in a ring, each with a reference bit. The hand sweeps forward: if a page's bit is 1, clear it and move on (that's the "second chance"); if 0, evict it. This approximates LRU without maintaining a full ordering, which is why real operating systems use it.

Cell 12b — Fast OPT

```

#cell 12b – Belady's optimal, with precomputed occurrence lists
class OPTFast:
    """
    Evicts the page whose next use is furthest in the future.
    ALLOWED to see the future – it is defined as the theoretical lower bound.
    """
    def __init__(self, pages=None, **kw):
        from collections import defaultdict
        import bisect
        self.occ = defaultdict(list)
        for i, p in enumerate(pages):
            self.occ[p].append(i)          # every index where page p appears
        self.bisect = bisect

    def on_insert(self, page, i): pass
    def on_hit(self, page, i): pass
    def on_evict(self, page): pass

    def choose_victim(self, resident, i):
        best, best_d = None, -1
        for p in resident:
            lst = self.occ.get(p, [])
            k = self.bisect.bisect_right(lst, i)  # first occurrence after i
            d = lst[k] if k < len(lst) else float('inf')
            if d > best_d:
                best, best_d = p, d
            if d == float('inf'):
                break          # never used again – evict now
        return best

```

Why the naive version is unusable

A straightforward OPT scans forward through the trace on every eviction to find each resident page's next use. That's $O(n)$ per eviction, and with 200,000 accesses it takes minutes per configuration.

OPTFast precomputes, for every page, a **sorted list of all indices where it appears**. Finding the next use after position `i` then becomes a binary search (`bisect_right`), which is $O(\log n)$.

The `break on float('inf')` is a further shortcut: a page never used again is the ideal victim, so there's no point examining the rest.

Why OPT is allowed to cheat

Every other policy receives the trace through `stream()` and cannot look ahead. OPT receives the full `pages` array in its constructor.

This asymmetry is deliberate and should be visible in your code. A reviewer reading it can see immediately that only OPT sees the future, which is exactly the clarity you want — it is the theoretical bound, not a competitor.

Cell 13 — Baseline comparison

```
#cell 13 – baseline comparison, frames scaled to working set
import pandas as pd, numpy as np

POLICIES = {'FIFO': FIFO, 'LRU': LRU, 'Clock': Clock, 'OPT': OPTFast}
RATIOS = [0.01, 0.02, 0.05, 0.10, 0.25] # fraction of working set

rows = []
for name, t in TRACES.items():
    ws = len(np.unique(t.pages)) # working set = unique pages
    for r in RATIOS:
        nf = max(8, int(ws * r))
        for pname, P in POLICIES.items():
            eng = MemoryEngine(nf, P(pages=t.pages))
            for i, page, _ in t.stream():
                eng.access(page, i)
            s = eng.stats()
            rows.append({'trace': name, 'ws': ws, 'ratio': r, 'frames': nf,
                        'policy': pname, 'faults': s['faults'],
                        'fault_rate': round(s['fault_rate'], 4)})

base = pd.DataFrame(rows)
base.to_csv(f'{BASE}/baseline_results.csv', index=False)
display(base.pivot_table(index=['trace', 'ratio', 'frames'], columns='policy',
                          values='fault_rate')[['FIFO', 'LRU', 'Clock', 'OPT']])
```

The critical design decision: scale frames to working set

This is the single most important choice in Stage 4, and getting it wrong wastes the whole stage.

A first attempt used fixed frame counts: 64, 128, 256, 512. The results were useless — bwaves showed **0.0157 at every single frame count**, with FIFO, LRU and Clock bitwise identical.

Why: bwaves has ~3,051 unique pages. At 512 frames its useful working set already fits, so nothing is ever evicted, so the eviction policy is **never exercised**. Meanwhile `cyclic` has only 20 unique pages — 512 frames is 25x its entire working set.

Memory pressure is what makes eviction decisions matter. With no pressure, all policies are identical and there is nothing to study.

Scaling frames as a *fraction of each trace's working set* puts every trace in a comparable regime. At 1–10% of working set, RAM holds only a small slice of the program's data, evictions happen constantly, and the policies separate.

The same logic applies to prefetching: if nothing is ever evicted, there is no cost to a wrong prefetch, and a cost-aware decision engine has nothing to weigh. Stage 5 experiments must run under real pressure or they will show nothing.

Known wrinkle

`cyclic` has a 20-page working set, so the `max(8, ...)` floor makes 1% and 2% both resolve to 8 frames, producing a duplicated and uninformative row. Either drop `cyclic` from this sweep or give it explicit frame counts such as `[4, 8, 12, 16, 20]`.

Cell 14 — Verification (do not skip)

```
#cell 14 – sanity checks on baselines
piv = base.pivot_table(index=['trace','frames'], columns='policy', values='faults')

fails = []

# 1. OPT must be the lower bound – nothing can beat it
for pol in ['FIFO','LRU','Clock']:
    bad = piv[piv[pol] < piv['OPT']]
    if len(bad): fails.append(f'{pol} beats OPT on {len(bad)} configs')

# 2. more frames must never increase faults (LRU/OPT are stack algorithms)
for name, t in TRACES.items():
    for pol in ['LRU','OPT']:
        sub = base[(base.trace==name)&(base.policy==pol)].sort_values('frames')
        if not sub.faults.is_monotonic_decreasing:
            fails.append(f'{pol} non-monotonic on {name}')

# 3. fault rate must be in [0,1]
if (base.fault_rate < 0).any() or (base.fault_rate > 1).any():
    fails.append('fault_rate out of range')

print('FAILED:' if fails else 'all checks passed')
for f in fails: print(' -', f)

# informative: how much better is LRU than FIFO?
cmp = piv.assign(LRU_vs_FIFO=lambda d: (d.FIFO-d.LRU)/d.FIFO)
display(cmp[['FIFO','LRU','OPT','LRU_vs_FIFO']].head(12))
```

What each check proves

Check 1 — OPT is the lower bound. Belady's algorithm is provably optimal. If any practical policy beats it, there is a bug — either in OPT's victim selection or in the engine. This is the strongest single correctness test available.

Check 2 — monotonicity. LRU and OPT are **stack algorithms**, meaning the set of pages resident with N frames is always a subset of the set resident with $N+1$ frames. A mathematical consequence: more memory can never cause more faults. Violation means a bug.

Note **FIFO is deliberately excluded** from this check. FIFO is *not* a stack algorithm and can exhibit **Belady's anomaly** — more frames producing more faults. That is a real, documented phenomenon, not a bug, and worth mentioning if you observe it.

Check 3 — range. Trivially catches arithmetic errors.

Part 16: What the results showed

The table structure

Each row is one experiment: "*trace X, N frames, policy P produced this fault rate.*" Four policy columns on the same trace and same memory means any difference is caused purely by the policy.

Verified results

All checks passed. Selected rows:

Trace	frames	FIFO	LRU	Clock	OPT
437.leslie3d-s0	21	0.0780	0.0766	0.0756	0.0294
437.leslie3d-s0	42	0.0167	0.0197	0.0166	0.0161
clustered	9	0.1146	0.1161	0.1135	0.0527
bfs-3	58	0.0348	0.0338	0.0340	0.0309
pr-3	40	0.0293	0.0291	0.0291	0.0267
410.bwaves-s0	30	0.0169	0.0157	0.0161	0.0156
sequential	any	1.0000	1.0000	1.0000	<1.0

Five findings

1. Fault rate falls as frames rise. The most basic sanity property, and it holds throughout.

2. Policies converge when memory is plentiful, separate when it is scarce. At 1% of working set the four columns differ meaningfully; at 25% they are often identical to four decimal places. This is the empirical justification for the ratio-based frame sweep.

3. sequential and strided at fault_rate exactly 1.0000 is correct, not broken. Those traces march through pages and never revisit one. Every access is to a never-before-seen page, so **every access must fault**, regardless of policy or memory size. OPT scores slightly below 1.0 only because it avoids pointless evictions. If any policy showed under 1.0 here, that would indicate a bug — so these traces are a useful anchor.

4. LRU is sometimes WORSE than FIFO. Observed twice:

Config	FIFO	LRU
clustered @ 9 frames	0.1146	0.1161

Config	FIFO	LRU
leslie3d @ 42 frames	0.0167	0.0197

Not a bug. In a tight random-access cluster, the least-recently-used page is very often the page about to be requested next — so LRU systematically evicts exactly the wrong thing, while FIFO's arbitrary choice accidentally does better.

This is the same underlying property that made clustered score only 0.099 useful in Stage 3. The workload is genuinely ambiguous, and cleverness based on recent history backfires. Two independent experiments pointing at the same conclusion is a strong thing to have in a paper.

5. Headroom varies enormously between configurations. The gap between LRU and OPT is how much a perfect policy would gain — and therefore roughly the ceiling on what prefetching could recover:

Trace	frames	LRU faults	OPT faults	headroom
437.leslie3d-s0	21	15,313	5,881	62%
bfs-3	58	6,766	6,179	9%
bfs-3	116	6,461	6,121	5%
410.bwaves-s0	30	3,140	3,112	1%
410.bwaves-s0	61+	3,135	3,051	~3%

leslie3d @ 21 frames is the showcase configuration — OPT causes less than 40% of LRU's faults. That is where prefetching has the most room to demonstrate benefit.

Conversely, **bwaves @ 30 frames has almost no headroom** (3,140 vs 3,112). No predictor, however good, can help much there. Running prefetching experiments in a no-headroom regime would show nothing and waste time.

Part 17: Choosing configurations for Stage 5

Based on the headroom analysis, Stage 5 experiments should target:

Trace	frames	Why
437.leslie3d-s0	21	62% headroom — best case for showing benefit
bfs-3	58, 116	Real reuse, moderate headroom, irregular workload
pr-3	40	Predictor does well here (0.892 useful)
clustered	9	Predictor does badly here (0.099 useful) — tests whether cost-awareness correctly declines
410.bwaves-s0	30	Low headroom control

The **clustered** configuration is the most important one. It is where a naive always-prefetch system should actively **hurt** performance, which is the empirical argument for a cost-aware decision engine.

Part 18: Stage 4 definition of done

- [] `MemoryEngine` defined; `_install()` is the only path into RAM
- [] FIFO, LRU, Clock, OPTFast all implement the four-method policy interface
- [] Baseline sweep uses frames **scaled to working set**, not fixed counts
- [] `baseline_results.csv` saved
- [] Cell 14 prints **"all checks passed"**
- [] OPT is the lowest fault count in every configuration
- [] LRU and OPT are monotonic in frame count
- [] `sequential / strided` show `fault_rate` exactly 1.0 for FIFO/LRU/Clock
- [] Headroom (LRU vs OPT) computed, and high-headroom configurations identified

Only when every box is ticked should the predictive layer be added. A bug found now costs minutes; the same bug found after Stage 5 is entangled with prediction logic and costs hours.

Part 19: Stage 4 artifacts

File	Goes into
<code>baseline_results.csv</code>	Results — classical policy comparison table

Plus the derived headroom analysis, which determines the experimental configurations for Stage 5.

Part 20: What Stage 4 does NOT do

No prediction. No prefetching. The `n_prefetch`, `n_pf_used` and `n_pf_wasted` counters all read zero.

What you have is a **verified measuring instrument**: a simulator whose baseline numbers can be trusted, and a quantified picture of where improvement is even possible.

Stage 5 adds the decision engine — the component that takes a prediction plus current system state and decides whether acting on it is worth the cost. The four configurations to compare there are:

1. **no-prefetch** (pure LRU — the baseline this stage established)
2. **always-prefetch** (naive, ignores cost — the control)
3. **fixed-threshold** (static confidence bar)
4. **cost-aware adaptive** (the proposed system)

Configuration 2 is the critical control. If always-prefetch performs **worse** than plain LRU on `clustered`, that proves prediction alone is insufficient — which is precisely the argument for the decision engine.

STAGE 5: The Decision Engine

What this stage does

Stage 3 built a predictor that says "*page 47 is next, 95% confident.*" Stage 4 built an engine that counts page faults.

Stage 5 connects them, and adds the component your project is actually about: the **decision engine**, which answers a different question from the predictor:

Is acting on this prediction worth it, right now?

This is the stage where "prediction accuracy" finally becomes "page faults avoided" — and where you discover that those are not the same thing.

Part 21: Core concepts

Why prediction and decision are separate components

They are connected, but deliberately never merged.

- **The predictor cannot see the system.** It knows access history. It has no idea how full RAM is, what would be evicted, or whether recent prefetches were wasted.
- **The decision engine cannot see the model.** It receives a number between 0 and 1. It knows nothing about Markov chains, contexts, or tables. Swap in an LSTM tomorrow and the decision engine needs zero changes.

The only interface between them is two values crossing a boundary: `(candidate_page, confidence)`.

This separation is what makes the ablation possible. Every decider in this stage runs against the *identical* predictor, so any difference in results is caused purely by the decision policy. If prediction and decision were one blob, you could never isolate the contribution of cost-awareness — and that contribution is the project's central claim.

Why a confident prediction can still be a bad idea

Prefetching is a bet:

- **Win:** you skip a page fault. Large payoff — disk is ~100,000x slower than RAM.
- **Loss:** you wasted I/O **and**, if RAM was full, evicted a page that might have been needed. A bad prefetch doesn't just fail to help — **it can cause a fault.**

The rule

Prefetch when (probability right x time saved) clearly exceeds (probability wrong x damage done).

In practice this becomes a **confidence threshold that moves with system conditions**:

Condition	Effect
Free frames available	A wrong prefetch costs nothing — be generous
RAM full	Every prefetch evicts something — demand more confidence
Recent prefetches paying off	Lower the bar
Recent prefetches failing	Raise the bar
Budget exhausted	Stop speculating — real work comes first

The feedback loop

Track whether prefetching is actually helping, and adjust the threshold. This is self-correcting and requires **no understanding of why** the workload changed. Its real value: a degraded predictor degrades gracefully into "do nothing" rather than actively hurting performance.

Choosing what to measure in this loop turned out to be the most important decision in the whole stage — see Part 23.

The ablation design

Six deciders, all consuming the same predictions:

Decider	Role
no-prefetch	Baseline — pure LRU from Stage 4
always	Critical control — acts on every prediction, ignores all cost
fixed-0.3 / fixed-0.6 / fixed-0.85	Static confidence bars
cost-aware	Adaptive, driven by prefetch precision
cost-aware-fd	Adaptive, driven by measured fault delta

`always` is not filler. If it performs worse than plain LRU somewhere, that proves prediction alone is insufficient — which is the argument for having a decision engine at all. Without that control, "cost-awareness helps" is an assertion; with it, it's a measurement.

Part 22: The code

Cell 14b — Diagnostic: where do predictions actually go?

Run this **before** designing thresholds. It tells you from data where the predictor's confidence actually sits, instead of guessing.

```
#cell 14b — where do predictions actually go?
import numpy as np, pandas as pd
```

```

def diagnose(tname, nf, thr=0.4, hist_len=8):
    t = TRACES[tname]
    pred = make_predictor('hybrid')
    eng = MemoryEngine(nf, LRU(pages=t.pages))
    hist = []
    n_pred = n_resident = n_lowconf = 0
    confs = []
    for i, page, _ in t.stream():
        eng.access(page, i)
        hist.append(page)
        if len(hist) > hist_len: hist.pop(0)
        if len(hist) >= 2: pred.update(hist[:-1], page)
        for c, cf in pred.predict(hist, top_n=1):
            n_pred += 1; confs.append(cf)
            if c in eng.resident: n_resident += 1
            elif cf < thr: n_lowconf += 1
    actionable = n_pred - n_resident - n_lowconf
    q = np.percentile(confs, [10,25,50,75,90]) if confs else [0]*5
    return {'trace': tname, 'frames': nf, 'accesses': len(t.pages),
            'predictions': n_pred, 'already_resident': n_resident,
            'below_thr': n_lowconf, 'actionable': actionable,
            'act_rate': round(actionable/max(1,len(t.pages)), 4),
            'conf_p10': round(q[0],3), 'conf_p25': round(q[1],3),
            'conf_p50': round(q[2],3), 'conf_p75': round(q[3],3),
            'conf_p90': round(q[4],3)}

CONFIGS = [('437.leslie3d-s0',21), ('bfs-3',58), ('bfs-3',116),
           ('pr-3',40), ('clustered',9), ('410.bwaves-s0',30)]

diag = pd.DataFrame([diagnose(tn, nf) for tn, nf in CONFIGS])
display(diag)

```

What the diagnostic revealed

Trace	frames	predictions	already resident	actionable	act_rate	conf_p25	conf_p50
437.leslie3d-s0	21	196,873	190,555	6,318	0.0316	0.927	0.961
bfs-3	58	193,701	193,275	426	0.0021	1.000	1.000
bfs-3	116	193,701	193,431	270	0.0014	1.000	1.000
pr-3	40	194,657	194,126	531	0.0027	1.000	1.000
clustered	9	49,058	44,460	4,598	0.0920	0.500	0.600
410.bwaves-s0	30	198,093	196,355	1,738	0.0087	0.875	0.957

Two findings that shaped everything after:

1. Most predictions are for pages already in RAM. On bfs-3 @58, 99.8% of predictions point at resident pages. There is nothing to prefetch — so the decision engine gets almost no opportunities, and every decider lands within a hair of baseline. This is why tighter memory configurations were added later.

2. Confidence is near-saturated. `below_thr` is **0 everywhere**, and `conf_p25` is 0.93–1.00 on most traces. The Markov chain is almost always maximally confident. **So confidence thresholds barely discriminate** — which is why `always` and `fixed-0.3` later produce bitwise-identical results. The discriminating factor had to be system state, not the predictor's self-reported certainty.

Cell 15 — Decision engines (final)

```
#cell 15 – prefetch decision policies

class NoPrefetch:
    name = 'no-prefetch'
    def decide(self, page, conf, state): return False
    def feedback(self, page, used): pass
    def stats(self): return {}

class AlwaysPrefetch:
    """Naive control: act on every prediction, ignore all cost."""
    name = 'always'
    def decide(self, page, conf, state): return True
    def feedback(self, page, used): pass
    def stats(self): return {}

class FixedThreshold:
    def __init__(self, thr=0.6):
        self.thr = thr
        self.name = f'fixed-{thr}'
    def decide(self, page, conf, state): return conf >= self.thr
    def feedback(self, page, used): pass
    def stats(self): return {'threshold': self.thr}

class CostAware:
    """
    Adaptive threshold driven by memory pressure and observed prefetch
    PRECISION.

    Differs from a fixed threshold in three ways:
    1. relaxes when free frames exist (a wrong prefetch costs nothing)
    2. refuses losing bets when RAM is full (conf <= 1-conf)
    3. moves the bar toward a TARGET precision band, updated on a throttle
       so the controller settles instead of slamming to a rail
    """
    name = 'cost-aware'

    def __init__(self, base_thr=0.5, lo=0.2, hi=0.9,
                 window=200, update_every=100, step=0.02,
                 target_lo=0.45, target_hi=0.75, budget_ratio=0.35):
        self.thr = base_thr
        self.lo, self.hi = lo, hi
        self.window = window
        self.update_every = update_every      # throttle: adjust every N events
        self.step = step
        self.target_lo = target_lo            # precision band to aim for
        self.target_hi = target_hi
        self.budget_ratio = budget_ratio
        self.recent = []
        self.thr_history = []
        self.n_feedback = 0
        self.d_budget = self.d_cost = self.d_thr = self.n_yes = 0

    def decide(self, page, conf, state):
        # 1. budget cap – speculation must not dominate real work
        if state['n_prefetch'] > self.budget_ratio * max(1, state['n_access']):
            self.d_budget += 1
            return False

        # 2. free frames -> a wrong prefetch costs nothing
        if state['free_frames'] > 0:
            ok = conf >= self.lo
            if ok: self.n_yes += 1
            else: self.d_cost += 1
```

```

        return ok

    # 3. RAM full -> benefit must exceed eviction damage
    if conf <= (1.0 - conf):          # conf <= 0.5 is a losing bet
        self.d_cost += 1
        return False

    # 4. adaptive bar
    if conf < self.thr:
        self.d_thr += 1
        return False

    self.n_yes += 1
    return True

def feedback(self, page, used):
    self.recent.append(1 if used else 0)
    if len(self.recent) > self.window:
        self.recent.pop(0)
    self.n_feedback += 1

    if self.n_feedback % self.update_every:    # throttle
        return
    if len(self.recent) < 50:
        return

    prec = sum(self.recent) / len(self.recent)
    if prec < self.target_lo: self.thr = min(self.hi, self.thr + self.step)
    elif prec > self.target_hi: self.thr = max(self.lo, self.thr - self.step)
    self.thr_history.append(self.thr)

def stats(self):
    th = self.thr_history
    return {'final_thr': round(self.thr, 3),
            'thr_min': round(min(th), 3) if th else None,
            'thr_max': round(max(th), 3) if th else None,
            'thr_moves': len(th),
            'approved': self.n_yes,
            'd_budget': self.d_budget,
            'd_cost': self.d_cost,
            'd_thr': self.d_thr}

print('decision engines ready')

```

The common interface

Every decider implements three methods. That uniformity is what lets the simulator treat them interchangeably:

Method	Purpose
<code>decide(page, conf, state)</code>	Return True to prefetch, False to skip
<code>feedback(page, used)</code>	Informed whether a past prefetch was used or wasted
<code>stats()</code>	Diagnostics for the results table

How `CostAware.decide()` makes a decision — the four gates

A candidate must pass every gate, in order:

1. **Budget** — if prefetches already exceed 35% of accesses, refuse. Speculation must never dominate real work.
2. **Free frames** — if RAM has space, a wrong prefetch evicts nothing, so accept anything above the low bar (0.2).

3. **Losing bet** — if RAM is full and `conf <= 1 - conf` (i.e. $\text{conf} \leq 0.5$), the expected damage exceeds the expected benefit. Refuse.
4. **Adaptive bar** — finally, confidence must exceed the current learned threshold.

The `update_every` throttle

An earlier version updated the threshold on **every** feedback event. It walked from 0.35 to a rail within ~20 events and never came back — every single run ended pinned at exactly 0.15 or 0.95. The controller had no stable operating point.

Throttling to every 100 events, with a *target band* (0.45–0.75) rather than a single trigger, lets it settle.

Cell 15b — Fault-delta cost-aware (the final design)

```
#cell 15b – cost-aware driven by measured fault delta, not precision

class CostAwareFaultDelta:
    """
    Adapts its confidence threshold using the ONLY signal that matters:
    is prefetching actually reducing faults relative to not prefetching?

    A shadow LRU engine runs the same trace with prefetching disabled.
    Every `epoch` accesses we compare fault counts:
        delta = shadow_faults - real_faults    (positive = prefetching helps)
    Threshold falls when prefetching is winning, rises when it is losing.
    """
    name = 'cost-aware-fd'

    def __init__(self, base_thr=0.5, lo=0.15, hi=0.95,
                  epoch=2000, step=0.05, budget_ratio=0.35,
                  dead_band=0.002):
        self.thr = base_thr
        self.lo, self.hi = lo, hi
        self.epoch = epoch          # accesses between adjustments
        self.step = step
        self.budget_ratio = budget_ratio
        self.dead_band = dead_band  # ignore deltas below this rate
        self.thr_history = []
        self.delta_history = []
        self.d_budget = self.d_cost = self.d_thr = self.n_yes = 0

    def decide(self, page, conf, state):
        if state['n_prefetch'] > self.budget_ratio * max(1, state['n_access']):
            self.d_budget += 1
            return False
        if state['free_frames'] > 0:
            ok = conf >= self.lo
            if ok: self.n_yes += 1
            else: self.d_cost += 1
            return ok
        if conf < self.thr:
            self.d_thr += 1
            return False
        self.n_yes += 1
        return True

    def epoch_update(self, real_faults, shadow_faults, n_access):
        """Called by the simulator every `epoch` accesses."""
        delta_rate = (shadow_faults - real_faults) / max(1, self.epoch)
        self.delta_history.append(round(delta_rate, 5))
        if delta_rate > self.dead_band:          # winning -> be bolder
            self.thr = max(self.lo, self.thr - self.step)
```

```

elif delta_rate < -self.dead_band:          # losing -> back off
    self.thr = min(self.hi, self.thr + self.step)
    self.thr_history.append(round(self.thr, 3))

def feedback(self, page, used):
    pass                                     # precision is NOT used

def stats(self):
    th, dh = self.thr_history, self.delta_history
    return {'final_thr': round(self.thr, 3),
            'thr_min': min(th) if th else None,
            'thr_max': max(th) if th else None,
            'thr_moves': len(th),
            'mean_delta_rate': round(np.mean(dh), 5) if dh else None,
            'epochs_winning': sum(1 for d in dh if d > 0),
            'epochs_losing': sum(1 for d in dh if d < 0),
            'approved': self.n_yes,
            'd_budget': self.d_budget, 'd_cost': self.d_cost,
            'd_thr': self.d_thr}

print('CostAwareFaultDelta ready')

```

The key idea: a shadow engine

To know whether prefetching is helping, you need to know how many faults would have happened *without* it. So the simulator runs a **second, identical LRU engine alongside the real one, with prefetching disabled**. Both see exactly the same accesses.

Every 2,000 accesses, compare:

```
delta = shadow_faults - real_faults
```

- **Positive** → prefetching is preventing faults → lower the threshold, be bolder
- **Negative** → prefetching is *causing* faults → raise the threshold, back off
- **Within the dead band** (± 0.002 per access) → noise → leave it alone

Note `feedback()` is a no-op. **Precision is deliberately ignored**. This decider only cares about the outcome that actually matters.

The honest limitation

A real operating system cannot run a shadow copy of itself. The shadow engine is an **experimental oracle** — it proves what the right feedback signal *is*, not that it can be cheaply obtained. A deployable version would need to estimate fault delta indirectly (e.g. by tracking whether evicted pages are re-faulted soon after a prefetch displaced them). **State this explicitly as a limitation in the paper**. It also roughly doubles runtime for this decider.

Cell 16 — Simulator with shadow support (final)

```

#cell 16 – simulator with optional shadow engine for fault-delta feedback
def run_prefetch_sim(trace, n_frames, policy_cls, predictor, decider,
                    hist_len=8, top_n=1):
    eng = MemoryEngine(n_frames, policy_cls(pages=trace.pages))
    history = []
    prev_wasted = 0

    # shadow engine: identical, but never prefetches
    uses_shadow = hasattr(decider, 'epoch_update')

```

```

shadow = MemoryEngine(n_frames, policy_cls(pages=trace.pages)) if uses_shadow else None

for i, page, _ in trace.stream():
    # --- 1. demand access ---
    was_pf = eng.prefetched.get(page, False)
    eng.access(page, i)
    if shadow is not None:
        shadow.access(page, i)
    if was_pf:
        decider.feedback(page, True)

    # --- 2. wasted prefetches ---
    if eng.n_pf_wasted > prev_wasted:
        for _ in range(eng.n_pf_wasted - prev_wasted):
            decider.feedback(None, False)
        prev_wasted = eng.n_pf_wasted

    # --- 3. epoch comparison (fault-delta feedback) ---
    if uses_shadow and eng.n_access % decider.epoch == 0:
        decider.epoch_update(eng.n_fault, shadow.n_fault, eng.n_access)

    # --- 4. update model ---
    history.append(page)
    if len(history) > hist_len:
        history.pop(0)
    if len(history) >= 2:
        predictor.update(history[:-1], page)

    # --- 5. predict + decide ---
    for cand, conf in predictor.predict(history, top_n=top_n):
        if cand in eng.resident:
            continue
        state = {'free_frames': eng.n_frames - len(eng.resident),
                 'n_frames': eng.n_frames,
                 'n_prefetch': eng.n_prefetch,
                 'n_access': eng.n_access,
                 'n_fault': eng.n_fault}
        if decider.decide(cand, conf, state):
            eng.prefetch(cand, i)

s = eng.stats()
s['decider'] = decider.name
s.update(decider.stats())
if shadow is not None:
    s['shadow_faults'] = shadow.n_fault
return s

print('simulator ready (with shadow support)')

```

The five steps on every access

1. **Demand access** — the correctness path. The real engine (and the shadow, if present) serves the request. If the page was prefetched, tell the decider it paid off.
2. **Wasted prefetches** — if any prefetched pages were evicted unused since the last step, report each as a failure.
3. **Epoch comparison** — every 2,000 accesses, the fault-delta decider compares real vs shadow faults.
4. **Update the model** — the Markov table learns the transition that just completed.
5. **Predict and decide** — get candidates, skip any already resident, and ask the decider about the rest.

Where the Markov chain and decision engine meet

```
for cand, conf in predictor.predict(history, top_n=top_n): # Markov speaks
    ...
    if decider.decide(cand, conf, state): # engine judges
        eng.prefetch(cand, i) # engine acts
```

That is the entire coupling. Two values cross the boundary; nothing else.

```
hasattr(decider, 'epoch_update')
```

The shadow engine is only created for deciders that need it. Every other decider runs at normal speed, and the simulator needs no special-casing beyond this one check.

Cell 17 — The ablation (final)

```
#cell 17 – ablation with fault-delta controller
import pandas as pd, numpy as np

CONFIGS = [
    ('437.leslie3d-s0', 21), # showcase – 62% headroom
    ('410.bwaves-s0', 30),
    ('clustered', 6),
    ('clustered', 9), # cost-awareness stress case
    ('bfs-3', 20),
    ('bfs-3', 58),
    ('pr-3', 15),
    ('pr-3', 40),
]

DECIDERS = [
    lambda: NoPrefetch(),
    lambda: AlwaysPrefetch(),
    lambda: FixedThreshold(0.3),
    lambda: FixedThreshold(0.6),
    lambda: FixedThreshold(0.85),
    lambda: CostAware(),
    lambda: CostAwareFaultDelta(),
]

EXTRA_KEYS = ('final_thr', 'thr_min', 'thr_max', 'thr_moves',
              'mean_delta_rate', 'epochs_winning', 'epochs_losing',
              'approved', 'd_budget', 'd_cost', 'd_thr', 'shadow_faults')

rows = []
for tname, nf in CONFIGS:
    t = TRACES[tname]

    e = MemoryEngine(nf, LRU(pages=t.pages))
    for i, p, _ in t.stream(): e.access(p, i)
    lru_ref = e.stats()['faults']

    e = MemoryEngine(nf, OPTFast(pages=t.pages))
    for i, p, _ in t.stream(): e.access(p, i)
    opt_ref = e.stats()['faults']

    headroom = (lru_ref - opt_ref) / max(1, lru_ref)

    for mk in DECIDERS:
        d = mk()
        s = run_prefetch_sim(t, nf, LRU, make_predictor('hybrid'), d)
        gain = (lru_ref - s['faults']) / max(1, lru_ref)
```

```

    row = {
        'trace': tname, 'frames': nf, 'decider': s['decider'],
        'faults': s['faults'],
        'vs_LRU': round(gain, 4),
        'headroom': round(headroom, 4),
        'prefetches': s['prefetches'],
        'pf_used': s['pf_used'],
        'pf_wasted': s['pf_wasted'],
        'pf_precision': round(s['pf_precision'], 3),
        'evictions': s['evictions'],
        'evict_vs_base': None,
        'LRU_ref': lru_ref, 'OPT_ref': opt_ref,
    }
    for k in EXTRA_KEYS:
        row[k] = s.get(k)
    rows.append(row)

abl = pd.DataFrame(rows)

# eviction inflation relative to no-prefetch for the same config
base_ev = (abl[abl.decider == 'no-prefetch']
            .set_index(['trace', 'frames'])['evictions'])
abl['evict_vs_base'] = abl.apply(
    lambda r: round(r.evictions / base_ev.loc[(r.trace, r.frames)], 3), axis=1)

abl.to_csv(f'{BASE}/ablation_results.csv', index=False)

print('=== fault reduction vs LRU (higher = better) ===')
display(abl.pivot_table(index=['trace', 'frames'], columns='decider', values='vs_LRU'))

print('\n=== prefetch precision (higher = less waste) ===')
display(abl.pivot_table(index=['trace', 'frames'], columns='decider', values='pf_precision'))

print('\n=== eviction inflation vs no-prefetch (lower = gentler) ===')
display(abl.pivot_table(index=['trace', 'frames'], columns='decider', values='evict_vs_base'))

# ===== SUMMARY: harm and benefit, honestly measured =====
sub = abl[abl.decider != 'no-prefetch'].copy()

summary = sub.groupby('decider').agg(
    mean_gain = ('vs_LRU', 'mean'),
    best_case = ('vs_LRU', 'max'),
    worst_case = ('vs_LRU', 'min'),
    total_harm = ('vs_LRU', lambda x: round(x[x < 0].sum(), 4)),
    total_gain = ('vs_LRU', lambda x: round(x[x > 0].sum(), 4)),
    n_configs = ('vs_LRU', 'size'),
).round(4).sort_values('worst_case', ascending=False)

print('\n=== decider summary (worst_case: closer to 0 = safer) ===')
display(summary)

# harm-only view
harmful_cfgs = (sub.groupby(['trace', 'frames'])['vs_LRU'].min() < 0)
harmful_cfgs = harmful_cfgs[harmful_cfgs].index
harm = (sub[sub.set_index(['trace', 'frames']).index.isin(harmful_cfgs)]
        .pivot_table(index=['trace', 'frames'], columns='decider', values='vs_LRU'))
print('\n=== configurations where prefetching can HURT ===')
display(harm)

# benefit-only view
good_cfgs = (sub.groupby(['trace', 'frames'])['vs_LRU'].max() > 0.05)
good_cfgs = good_cfgs[good_cfgs].index
good = (sub[sub.set_index(['trace', 'frames']).index.isin(good_cfgs)]
        .pivot_table(index=['trace', 'frames'], columns='decider', values='vs_LRU'))
print('\n=== configurations where prefetching clearly HELPS ===')
display(good)
print('\nbenefit retained vs best decider:')
display((good / good.max(axis=1).values.reshape(-1,1)).round(4))

```

```
# fault-delta controller behaviour
print('\n=== fault-delta controller: did it detect harm correctly? ===')
cols = ['trace', 'frames', 'final_thr', 'thr_min', 'thr_max', 'thr_moves',
        'mean_delta_rate', 'epochs_winning', 'epochs_losing',
        'prefetches', 'pf_precision', 'vs_LRU']
fd = abl[abl.decider=='cost-aware-fd']
fdv = fd[[c for c in cols if c in fd.columns]].copy()
fdv['verdict'] = np.where(fdv.epochs_losing > fdv.epochs_winning,
                        'shut down', 'went aggressive')

display(fdv)

# precision vs benefit
print('\n=== precision vs benefit ===')
pv = sub[['trace', 'frames', 'decider', 'pf_precision', 'vs_LRU', 'evict_vs_base']].copy()
corr = pv[['pf_precision', 'vs_LRU']].corr().iloc[0,1]
print(f'correlation(precision, fault reduction) = {corr:.3f}')
ex = pv[(pv.pf_precision > 0.75) & (pv.vs_LRU < 0)]
print(f'configs with HIGH precision (>0.75) but NEGATIVE benefit: {len(ex)}')
display(ex)

summary.to_csv(f'{BASE}/decider_summary.csv')
print('\nfull detail:')
display(abl)
```

Why the configurations were chosen

Config	Purpose
leslie3d @21	Showcase — 62% LRU-to-OPT headroom
bwaves @30	Regular workload, good predictor accuracy
clustered @6, @9	Predictor is <i>bad</i> here — tests whether cost-awareness correctly declines
bfs-3 @20, @58	Irregular graph workload; @20 added because @58/@116 had almost no opportunity
pr-3 @15, @40	Irregular, but predictor does reasonably

The original bfs-3 @116 was dropped: its `act_rate` was 0.0014 — only 270 actionable predictions in 200,000 accesses. Tighter memory configurations create evictions, which create prefetch opportunities.

EXTRA_KEYS — why rows are built this way

Each decider returns different diagnostic keys from `stats()`. An earlier version built rows from a fixed dict and silently dropped the fault-delta keys, causing a `KeyError` in the diagnostic table. Looping over `EXTRA_KEYS` with `s.get(k)` fills in `None` for any decider that doesn't report a given key.

evict_vs_base — eviction inflation

Evictions under each decider, divided by evictions with no prefetching on the same config. A value of 1.785 means that decider caused 78.5% more evictions than plain LRU. **This is where the cost of bad prefetching becomes visible** — every extra eviction is a page displaced from RAM, potentially one that was about to be needed.

Why `total_harm` instead of counting harmful configs

A first summary counted configurations with `vs_LRU < -0.001`. That was misleading: a decider that nearly shut prefetching off still got counted as "harmful" for a `-0.0028` result, while another scored better only because it happened to land at `-0.0005` once. **Magnitude matters, not the count.** `total_harm` sums all negative outcomes.

Part 23: How the decision engine evolved

This stage went through four versions. The failures are as instructive as the final design, and worth describing in the paper.

Version 1 — cost-aware collapsed onto fixed-threshold

The first `CostAware` produced output **bitwise identical to fixed-0.6** on multiple configurations. Two causes: adaptation required 50 feedback samples before engaging, and the feedback hook only fired on *successful* prefetches, never wasted ones. On traces issuing only 84 prefetches all run, it never adapted at all.

Version 2 — the controller slammed to the rails

After fixing the feedback, every run ended with the threshold pinned at exactly **0.15 or 0.95**:

trace	final_thr	thr_moves
leslie3d	0.15 (floor)	5,588
bfs-3 @58	0.95 (ceiling)	33
pr-3	0.95 (ceiling)	322
clustered	0.15 (floor)	2,204

Updating on every event with `step=0.03` walked it to a rail in ~20 events. No stable operating point.

Version 3 — precision-based, throttled

Adding the `update_every=100` throttle and a target band made the threshold settle between the rails (range 0.20–0.64). **The controller now worked mechanically — but it produced the worst result in the entire table:**

On `clustered @9`, the configuration designed specifically to showcase cost-awareness, precision-based cost-aware scored **−0.0122 — worse than the naive always control (+0.0009)**.

The reason: precision on that config was 0.81 — prefetched pages were being used. So the controller concluded prefetching was working and lowered its threshold to the floor (0.20). But evictions had risen 39.6% over baseline. **The prefetched pages were used, but installing them displaced other pages that were needed sooner.**

Precision measures whether a prefetch was used. It does not measure whether it helped.

Version 4 — fault-delta (final)

Replace the proxy with the real target. Stop asking "*were my prefetches used?*" and ask "*am I causing fewer faults than I would without prefetching?*" — measured directly against the shadow engine.

Part 24: Results

Fault reduction vs LRU (higher = better)

Trace	frames	always	fixed-0.3	fixed-0.6	fixed-0.85	cost-aware	cost-aware-fd
410.bwaves-s0	30	0.3729	0.3729	0.3729	0.3729	0.3729	0.3729
437.leslie3d-s0	21	0.3500	0.3500	0.3359	0.2890	0.3413	0.3498
bfs-3	20	−0.0150	−0.0150	−0.0041	−0.0005	−0.0045	−0.0028
bfs-3	58	−0.0027	−0.0027	−0.0006	0.0001	−0.0006	−0.0019
clustered	6	−0.0087	−0.0087	−0.0098	−0.0081	−0.0096	−0.0072
clustered	9	0.0009	0.0009	−0.0093	−0.0019	−0.0122	−0.0048
pr-3	15	0.0051	0.0051	0.0063	0.0059	0.0063	0.0051
pr-3	40	0.0074	0.0074	0.0069	0.0067	0.0069	0.0074

Raw fault counts

Trace	frames	no-prefetch	always	cost-aware	cost-aware-fd	OPT (no prefetch)
437.leslie3d-s0	21	15,313	9,954	10,086	9,956	5,881
410.bwaves-s0	30	3,140	1,969	1,969	1,969	3,112
clustered	9	5,803	5,798	5,874	5,831	2,637
clustered	6	20,344	20,522	20,539	20,490	10,361
bfs-3	20	7,394	7,505	7,427	7,415	6,349

Eviction inflation vs no-prefetch (lower = gentler)

Trace	frames	always	fixed-0.6	fixed-0.85	cost-aware	cost-aware-fd
clustered	6	1.933	1.456	1.215	1.448	1.279
clustered	9	1.785	1.383	1.164	1.396	1.236
bfs-3	20	1.149	1.036	1.013	1.039	1.029
410.bwaves-s0	30	1.054	1.037	1.024	1.049	1.054
437.leslie3d-s0	21	1.032	1.025	1.022	1.025	1.032

On clustered @6, naive prefetching nearly **doubles** evictions (1.933x). Fault-delta holds it to 1.279x.

Decider summary

Decider	mean gain	best case	worst case	total harm	total gain
cost-aware-fd	0.0898	0.3729	−0.0072	−0.0167	0.7352
fixed-0.85	0.0830	0.3729	−0.0081	−0.0105	0.6746
fixed-0.6	0.0873	0.3729	−0.0098	−0.0238	0.7220
cost-aware	0.0876	0.3729	−0.0122	−0.0269	0.7274
always	0.0887	0.3729	−0.0150	−0.0264	0.7363
fixed-0.3	0.0887	0.3729	−0.0150	−0.0264	0.7363

Fault-delta has the **highest mean gain and the best worst case** of any decider.

Benefit retained on favourable workloads

Trace	frames	always	fixed-0.85	cost-aware	cost-aware-fd
410.bwaves-s0	30	1.0000	1.0000	1.0000	1.0000
437.leslie3d-s0	21	1.0000	0.8257	0.9751	0.9994

Fault-delta controller behaviour

Trace	frames	final_thr	range	epochs winning	epochs losing	verdict
437.leslie3d-s0	21	0.15	0.15-0.45	100	0	went aggressive ✓
410.bwaves-s0	30	0.15	0.15-0.45	100	0	went aggressive ✓
clustered	6	0.95	0.55-0.95	1	24	shut down ✓
clustered	9	0.95	0.50-0.95	0	25	shut down ✓
bfs-3	20	0.95	0.40-0.95	5	29	shut down ✓
bfs-3	58	0.85	0.15-0.85	16	17	oscillating
pr-3	15	0.45	0.45-0.50	2	0	mildly aggressive
pr-3	40	0.40	0.40-0.50	2	0	mildly aggressive

This table is the direct evidence that the adaptive mechanism works. It detected benefit in every favourable configuration and drove the threshold to the floor. It detected harm in every harmful configuration and drove the threshold to the ceiling. Not asserted — shown, with the control signal and the response side by side.

Precision vs benefit

Correlation between prefetch precision and fault reduction across all runs: **+0.73**.

Configurations with high precision (>0.75) but negative benefit:

Trace	frames	decider	precision	vs_LRU	eviction inflation
clustered	9	fixed-0.6	0.822	−0.0093	1.383
clustered	9	fixed-0.85	0.863	−0.0019	1.164
clustered	9	cost-aware	0.812	−0.0122	1.396
clustered	9	cost-aware-fd	0.845	−0.0048	1.236

Within that single configuration, **eviction inflation tracks the outcome almost perfectly** (higher inflation → worse result), while precision does not.

Part 25: Findings

These are the claims the data supports. State them in this form in the paper.

Finding 1 — Prefetching gives large gains on regular workloads

35-37% fault reduction on leslie3d (15,313 → 9,956) and bwaves (3,140 → 1,969), under real memory pressure, with prefetch precision of 0.88-0.95.

Finding 2 — Confidence thresholds do not discriminate

`always` and `fixed-0.3` are **bitwise identical on every configuration**. The Markov chain's confidence is near-saturated (`conf_p25` = 0.93-1.00 on most traces; zero predictions below threshold anywhere). A threshold on the predictor's self-reported certainty cannot separate good predictions from bad ones when nearly all of them report ≥ 0.93 .

The discriminating signal must come from system state, not from the predictor.

Finding 3 — Precision-based adaptation is counterproductive

Precision-driven cost-aware had a worse worst case (−0.0122) than doing nothing adaptive at all, **and** retained less benefit on leslie3d (97.5%) than the naive control. It lost on both axes.

Finding 4 — Fault-delta adaptation gives the best safety/benefit trade-off

	<code>always</code>	<code>cost-aware-fd</code>
Worst case	−0.0150	−0.0072
Benefit retained (leslie3d)	100%	99.94%

Halves worst-case harm while keeping 99.94% of the benefit. Also the highest mean gain of any decider (0.0898).

Compare `fixed-0.85`: lower *total* harm (−0.0105) but retains only **82.6%** of the benefit on leslie3d. A conservative fixed threshold is safe by being timid everywhere; fault-delta is safe only where it needs to be.

Finding 5 — High precision can coexist with net harm

Precision correlates positively with benefit overall (+0.73), so it is not useless. But it **fails in a specific regime**: under high eviction pressure, a prefetched page can be used and still be net-harmful, because installing it displaced a page that was needed sooner. Four deciders on clustered @9 achieved 0.81-0.86 precision while all producing negative outcomes.

A correction worth noting. An earlier informal reading of these results claimed precision was *uncorrelated* with benefit. That was wrong — the measured correlation is +0.73. The accurate claim is narrower: precision is a reasonable global signal but is **insufficient when eviction pressure is high**. Say it that way.

Finding 6 — Prefetching can beat the no-prefetch OPT bound

bwaves @30: prefetching reaches **1,969 faults** while OPT reaches **3,112**. This is not a bug and it does not contradict OPT's optimality.

OPT is optimal among *replacement* policies — it chooses which page to evict, but it still has to fault on every page's first access (compulsory faults). **Prefetching avoids compulsory faults** by loading pages before their first request, which no replacement policy can do. OPT here is the bound for a *non-prefetching* system.

Report `pct_of_headroom` carefully: it exceeds 100% on bwaves for this reason. Explain it rather than reporting a confusing number.

Part 26: Limitations to state honestly

1. **The shadow engine is an oracle.** A real OS cannot run a copy of itself. Fault-delta proves what the right feedback signal is; it does not show that signal can be obtained cheaply in deployment.
2. **Gains are concentrated in two configurations.** Large benefits appear on leslie3d and bwaves. On the four irregular configurations, every decider lands within $\pm 1.5\%$ of baseline. The decision engine's value there is *limiting harm*, not producing gain.
3. **bfs-3 @58 oscillates.** 16 winning epochs, 17 losing. The delta rate (-0.00038) sits inside the dead band much of the time, so the controller never settles. Fault-delta also scores worse than fixed-0.85 there (-0.0019 vs $+0.0001$). A wider dead band or longer epoch might help; untested.
4. **The dead band and epoch are untuned.** `epoch=2000`, `dead_band=0.002`, `step=0.05` were chosen once and not swept. They should be tuned on training data only, then frozen — the same discipline as the Stage 3 hyperparameters.
5. **Single run per configuration.** The real traces are deterministic, but results have not been checked across trace segments (`-s1` , `-s2`) or across seeds for the synthetic traces. That belongs in Stage 6.
6. **LLC traces at page granularity.** Inherited from Stage 1 — applies to every result.

Part 27: Stage 5 definition of done

- [] Cell 14b diagnostic run; confidence distribution recorded
- [] All six deciders implement `decide` / `feedback` / `stats`
- [] Simulator reports wasted prefetches to the decider, not just used ones
- [] Shadow engine created only for deciders with `epoch_update`
- [] `ablation_results.csv` and `decider_summary.csv` saved
- [] `always` shows negative `vs_LRU` on at least one config (the control works)
- [] Precision-based cost-aware threshold settles off the rails
- [] Fault-delta controller shuts down on harmful configs and goes aggressive on favourable ones
- [] Limitations written down

Part 28: Stage 5 artifacts

File	Goes into
ablation_results.csv	Results — the main four-way comparison
decider_summary.csv	Results — safety/benefit trade-off table

Part 29: What comes next — Stage 6

Stage 5 answered the core research question. Stage 6 makes the answer robust and presentable:

- **Figures** — fault reduction by decider, eviction inflation, the fault-delta threshold trajectory over time, precision-vs-benefit scatter
- **Robustness** — repeat on `-s1` / `-s2` trace segments; multiple seeds on synthetic traces; report mean $\pm$ standard deviation
- **Sensitivity** — sweep the fault-delta `epoch` and `dead_band` on training data only, then freeze
- **Frame sweep** — run the best deciders across the full 1-25% working-set range, not just hand-picked points

Appendix: Common errors

Error	Cause	Fix
<code>NameError: name 'OUT' is not defined</code>	Runtime restarted	Re-run SETUP cell
<code>NameError: name 'convert' is not defined</code>	Same	Re-run converter cell
<code>grep: download_links: No such file</code>	Repo clone wiped	Re-clone ChampSim
skipped in the thousands	Wrong delimiter or column	Check with <code>!head -5</code> on the raw file
<code>BAD_DOWNLOAD</code>	Box returned an HTML error page	Download via browser to Drive instead
<code>reuse ≈ 1.0</code>	Page shift too small, or wrong column	Verify <code>addr_col=2</code> ; try larger <code>PAGE_SHIFT</code>
Only one unique page	Grabbed the hit/miss column	Confirm <code>addr_col=2</code>
Disk full	<code>.xz</code> files accumulating in <code>/content</code>	<code>!rm -f /content/*.txt.xz</code>
<code>'list' object has no attribute 'pages'</code>	<code>TRACES</code> holds raw lists	Re-run the "rebuild TRACES" cell
<code>NameError: name 'FROZEN' is not defined</code>	Runtime restarted, or FROZEN cell not run	Re-run the FROZEN cell
<code>_IncompleteInputError: incomplete input</code>	Code cell was pasted truncated	Clear the cell fully and re-paste
Results differ between runs	Two configs in play (e.g. a stray <code>CAP</code>)	Ensure only <code>make_predictor()</code> builds predictors

Error	Cause	Fix
Table shows 0.000 for Markov on <code>sequential</code>	Working set > table capacity	Expected; stride fallback covers it
Memory footprint absurdly large (~1 MB)	Used <code>sys.getsizeof()</code>	Use the information-theoretic estimate
Cost-aware identical to fixed-threshold	Adaptation never engages; feedback only on successes	Report wasted prefetches too; lower <code>min_samples</code>
Threshold pinned at 0.15 or 0.95 every run	Updating on every event	Throttle with <code>update_every</code> ; use a target band
<code>always</code> and <code>fixed-0.3</code> bitwise identical	Confidence near-saturated (<code>conf_p25</code> $\approx 0.93+$)	Expected — confidence alone cannot discriminate
All deciders within ± 0.003 of baseline	Almost every prediction already resident	Use tighter memory (check <code>act_rate</code> in Cell 14b)
<code>KeyError: ['mean_delta_rate', ...] not in index</code>	Row dict drops decider-specific stats	Loop over <code>EXTRA_KEYS</code> with <code>s.get(k)</code>
Prefetching beats OPT	OPT bounds replacement only; prefetch avoids compulsory faults	Expected — explain, don't hide
High precision but negative benefit	Used prefetches displaced pages needed sooner	Use fault-delta feedback, not precision

Appendix B: Key numbers at a glance

Reference values from the completed Stages 1-3, for the paper.

Configuration

Parameter	Value
Page size	4 KB (<code>PAGE_SHIFT = 12</code>)
Trace cap	200,000 accesses
Warm-up fraction	0.25
Markov capacity	256 contexts
Successors per context	2
Markov order	2
Decay interval	25,000 updates

Predictor memory

Metric	Value
Bounded ceiling	8.5 KB (~2.1 pages)
Overhead @ 128 frames	1.66%
Overhead @ 1024 frames	0.21%

Headline results (hybrid predictor, `useful` = coverage x accuracy)

Trace	useful
410.bwaves-s0	0.804
437.leslie3d-s0	0.844
bfs-3	0.706
pr-3	0.892
clustered	0.099
random	0.000

Mean across real traces: **0.783** | OPT runs for minutes per config | Naive O(n) next-use scan | Use OPTFast with precomputed occurrence lists | | All policies give identical fault rates | Fixed frame counts, no memory pressure | Scale frames to each trace's working set | | A policy beats OPT | Bug in OPT or in the engine | Do not proceed; OPT is provably optimal | | LRU non-monotonic in frames | Bug (LRU is a stack algorithm) | Do not proceed | | FIFO non-monotonic in frames | Belady's anomaly | Expected, not a bug | | `fault_rate` exactly 1.0 on sequential | Every page touched once, never reused | Expected, not a bug |

Stage 4 baselines — headroom (LRU vs OPT)

Trace	frames	LRU faults	OPT faults	headroom
437.leslie3d-s0	21	15,313	5,881	62%
bfs-3	58	6,766	6,179	9%
bfs-3	116	6,461	6,121	5%
410.bwaves-s0	30	3,140	3,112	1%

Frame sweep: 1%, 2%, 5%, 10%, 25% of each trace's working set. Verification: all checks passed (OPT lower bound, LRU/OPT monotonic, rates in range).

Stage 5 — decision engine results

Decider	mean gain	worst case	total harm	benefit retained (leslie3d)
cost-aware-fd	0.0898	-0.0072	-0.0167	99.94%
fixed-0.85	0.0830	-0.0081	-0.0105	82.57%
cost-aware (precision)	0.0876	-0.0122	-0.0269	97.51%
always	0.0887	-0.0150	-0.0264	100%

Headline	Value
Fault reduction, leslie3d @21	35.0% (15,313 → 9,956)
Fault reduction, bwaves @30	37.3% (3,140 → 1,969)
Worst-case harm, always → fault-delta	-0.0150 → -0.0072 (halved)
Precision vs benefit correlation	+0.73
Fault-delta epoch / dead band / step	2000 / 0.002 / 0.05 (untuned)